EXPERIMENT NO 1

TITLE:
Generation of Basic Discrete-Time Signals Using MATLAB.

AIM:
To generate and study the following discrete-time signals using MATLAB using conditional statements and looping constructs:

1. Unit Impulse Signal
2. Unit Step Signal
3. Ramp Signal
4. Positive Exponential Signal
5. Negative Exponential Signal
6. Double-Sided Exponentially Growing Signal
7. Double-Sided Exponentially Decaying Signal
8. Discrete Sine
9. Discrete Cosine

HARDWARE REQUIRED:

Personal Computer (PC) / Laptop

SOFTWARE / SIMULATION TOOLS REQUIRED:

MATLAB Software (R2018a or Higher)

THEORY:

A discrete-time signal is defined only at discrete instants of time and is represented as a sequence of numbers. Basic discrete-time signals form the building blocks for analyzing and designing digital signal processing systems.

1. Unit Impulse Signal

$\delta(n) = 1, n = 0$
$\delta(n) = 0, n \neq 0$

The signal contains a single non-zero sample at n = 0.

2. Unit Step Signal

$u(n) = 1, n \geq 0$
$u(n) = 0, n < 0$

The signal remains zero for negative indices and one for positive indices.

3. Ramp Signal

   $r(n) = n\,u(n)$

   The amplitude increases linearly with time.

4. Positive Exponential Signal

   $x(n) = a^n$, $a > 1$

   The signal grows exponentially as n increases.

5. Negative Exponential Signal

   $x(n) = a^n$, $0 < a < 1$

   The signal decays exponentially as n increases.

6. Double-Sided Exponentially Growing Signal

   $x(n) = a^{|n|}$, $a > 1$

   The signal grows symmetrically on both sides of the origin.

7. Double-Sided Exponentially Decaying Signal

   $x(n) = a^{|n|}$, $0 < a < 1$

   The signal decays symmetrically on both sides of the origin.

8. Discrete Sinusoidal Signal:

$$x(n) = A\sin(\omega n)$$

Discrete Cosine Signal:

$$x(n) = A\cos(\omega n)$$

where: A= Amplitude,ω= Angular frequency in radians/sample , n= Discrete-time index

ALGORITHM:

Step 1: Start the program.

Step 2: Read the starting and ending values of n.

Step 3: Generate the unit impulse sequence using a for loop and if condition.

Step 4: Generate the unit step sequence using a for loop and if condition.

Step 5: Generate the ramp sequence using a for loop and if condition.

Step 6: Read the value of a (>1) and generate the positive exponential sequence.

Step 7: Read the value of b (0<b<1) and generate the negative exponential sequence.

Step 8: Read the value of c (>1) and generate the double-sided exponentially growing sequence.

Step 9: Read the value of d (0<d<1) and generate the double-sided exponentially decaying sequence.

Step 10: Read the value of Amplitude of sine signal, Angular frequency

Step 11: Read the value of Amplitude of Cosine signal, Angular frequency

Step 12: Plot all signals using stem().

Step 13: Display the outputs.

Step 14: Stop the program.

MATLAB Program:

```
% EXPERIMENT:
% Generation of Basic Discrete-Time Signals Using MATLAB

clc; % Clear command window
clear; % Clear workspace
close all; % Close all figures

% Read range of n from user
n1 = input('Enter starting value of n = ');
n2 = input('Enter ending value of n = ');

% Generate index vector
n = n1:n2;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1. UNIT IMPULSE SIGNAL
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

for i = 1:length(n)

if n(i) == 0

   x1(i) = 1;

else

   x1(i) = 0;

end

end
```

```
figure;
stem(n,x1,'filled');
title('Unit Impulse Signal');
xlabel('n');
ylabel('Amplitude');
grid on;

%%%
```

```
title('Ramp Signal');
xlabel('n');
ylabel('Amplitude');
grid on;

%%%
```

```
x6(i) = c^(abs(n(i)));

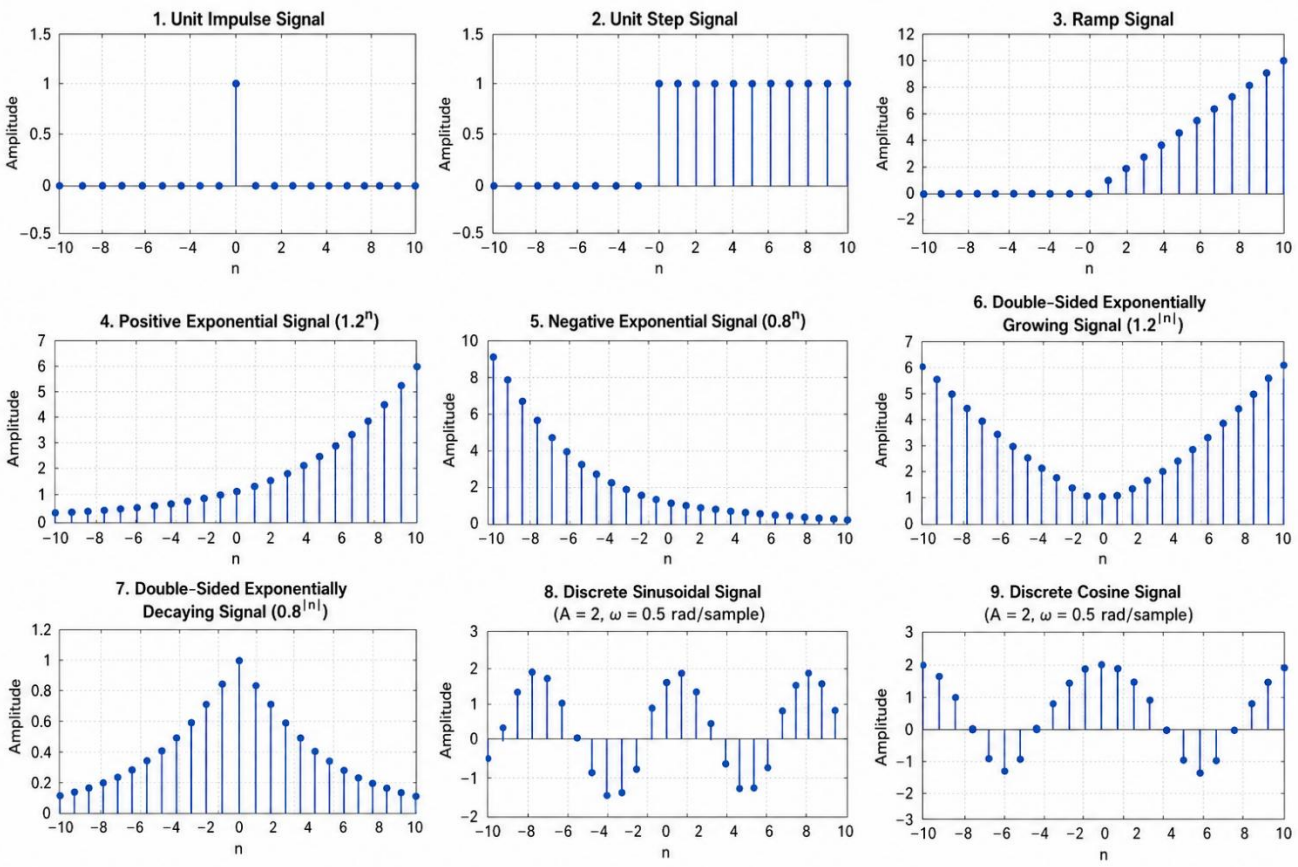
end

figure;
stem(n,x6,'filled');
title('Double Sided Exponentially Growing Signal');
xlabel('n');
ylabel('Amplitude');
grid on;

%%%
```

```
%%%
```

Sample Output



Result:

Thus, the Unit Impulse, Unit Step, Ramp, Positive Exponential, Negative Exponential, Double-Sided Exponentially Growing, Double-Sided Exponentially Decaying, Discrete Sinusoidal, and Discrete Cosine signals were successfully generated and plotted using MATLAB.

Experiment 2 a) : Linear Convolution

Aim

To write a MATLAB program to perform the linear convolution of two discrete-time sequences without using the built-in conv() function and to plot the input and output sequences.

Simulation Tools / Hardware Required

Software

MATLAB R2021a or later (any recent version)

Hardware

Personal Computer/Laptop

Theory

Linear convolution is a mathematical operation used to determine the output of a **Linear Time-Invariant (LTI)** system for a given input sequence.

If

$$x(n) = \{x_0, x_1, \ldots, x_{N-1}\}$$

and

$$h(n) = \{h_0, h_1, \ldots, h_{M-1}\}$$

then the linear convolution is

$$y(n) = x(n) * h(n)$$

where

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k)$$

The length of the output sequence is

$$L = N + M - 1$$

In this experiment, the convolution is performed manually using **nested for-loops**, without using MATLAB's built-in **conv()** function.

The starting index of both sequences may be any integer (positive, zero, or negative). Accordingly, the output sequence starting index is

$$n_y = n_x + n_h$$

where

= Starting index of input sequence , = Starting index of impulse response

Algorithm

1. Start the program.
2. Enter the first discrete sequence.
3. Enter its starting index.
4. Enter the second discrete sequence.
5. Enter its starting index.
6. Find the lengths of both sequences.
7. Initialize the output sequence with zeros.
8. Perform linear convolution using two nested loops.
9. Compute the output starting index.
10. Generate the corresponding index values.
11. Plot the first input sequence.
12. Plot the second input sequence.
13. Plot the output sequence.
14. Display the convolution result.
15. Stop.

Program

```
clc;
clear;
close all;

% Input first sequence
x = input('Enter the first sequence x(n): ');
sx = input('Enter the starting index of x(n): ');

% Input second sequence
h = input('Enter the second sequence h(n): ');
sh = input('Enter the starting index of h(n): ');

N = length(x);
M = length(h);

% Output length
L = N + M - 1;
```

```
% Initialize output
y = zeros(1,L);

% Linear convolution without built-in function
for i = 1:N
   for j = 1:M
      y(i+j-1) = y(i+j-1) + x(i)*h(j);
   end
end

% Index generation
nx = sx : sx+N-1;
nh = sh : sh+M-1;
sy = sx + sh;
ny = sy : sy+L-1;

disp('Linear Convolution Output = ');
disp(y);

% Plotting
figure;
subplot(3,1,1);
stem(nx,x,'filled','LineWidth',2);
grid on;
xlabel('n');
ylabel('Amplitude');
title('Input Sequence x(n)');

subplot(3,1,2);
stem(nh,h,'filled','LineWidth',2);
grid on;
xlabel('n');
ylabel('Amplitude');
title('Input Sequence h(n)');
```

```
subplot(3,1,3);
stem(ny,y,'filled','LineWidth',2);
grid on;
xlabel('n');
ylabel('Amplitude');
title('Linear Convolution Output y(n)');
```

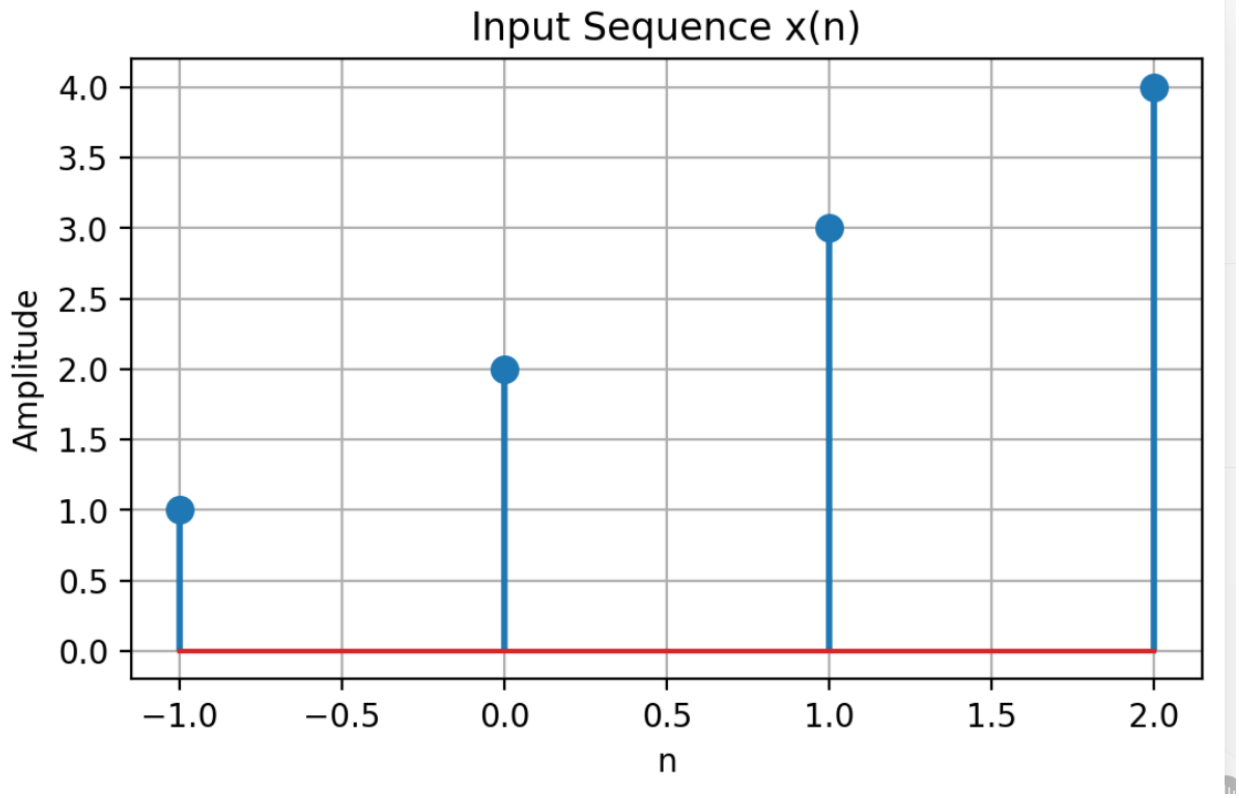
Sample Input

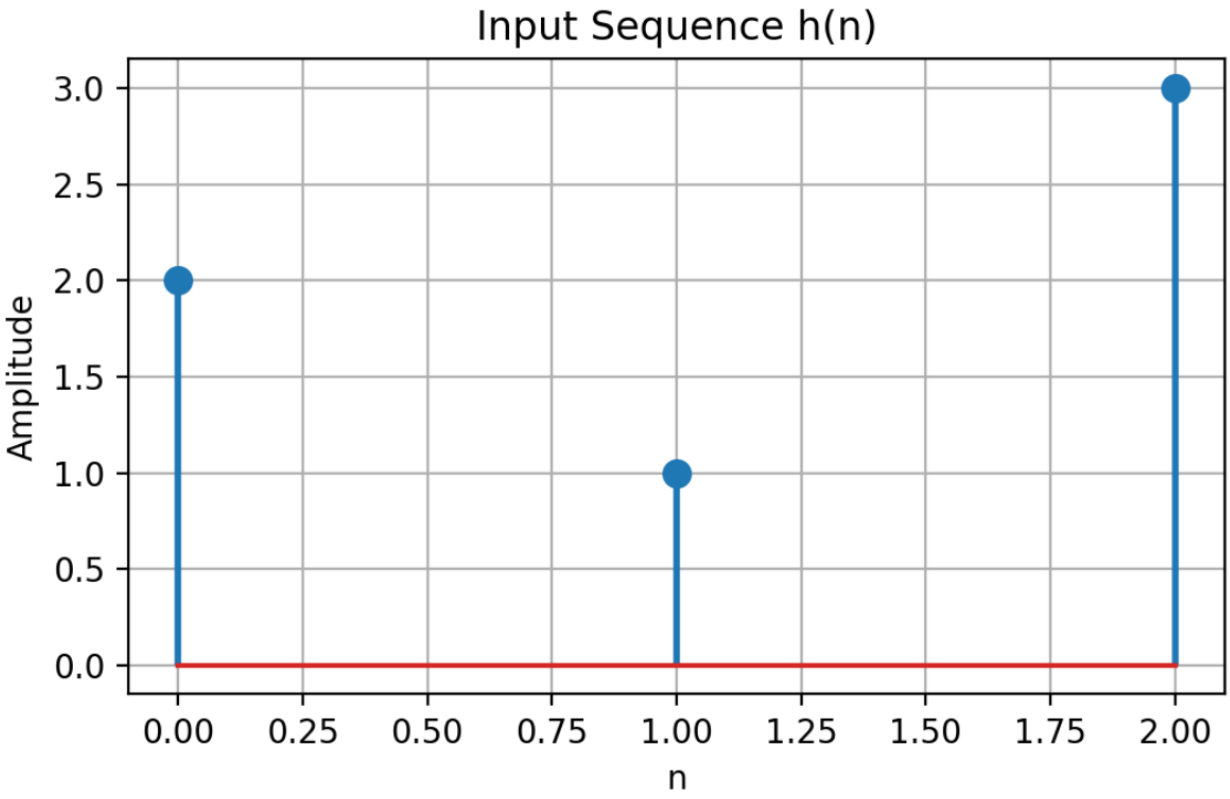
```
Enter the first sequence x(n):
[1 2 3 4]
Enter the starting index of x(n):
-1
Enter the second sequence h(n):
[2 1 3]
Enter the starting index of h(n):
0
```

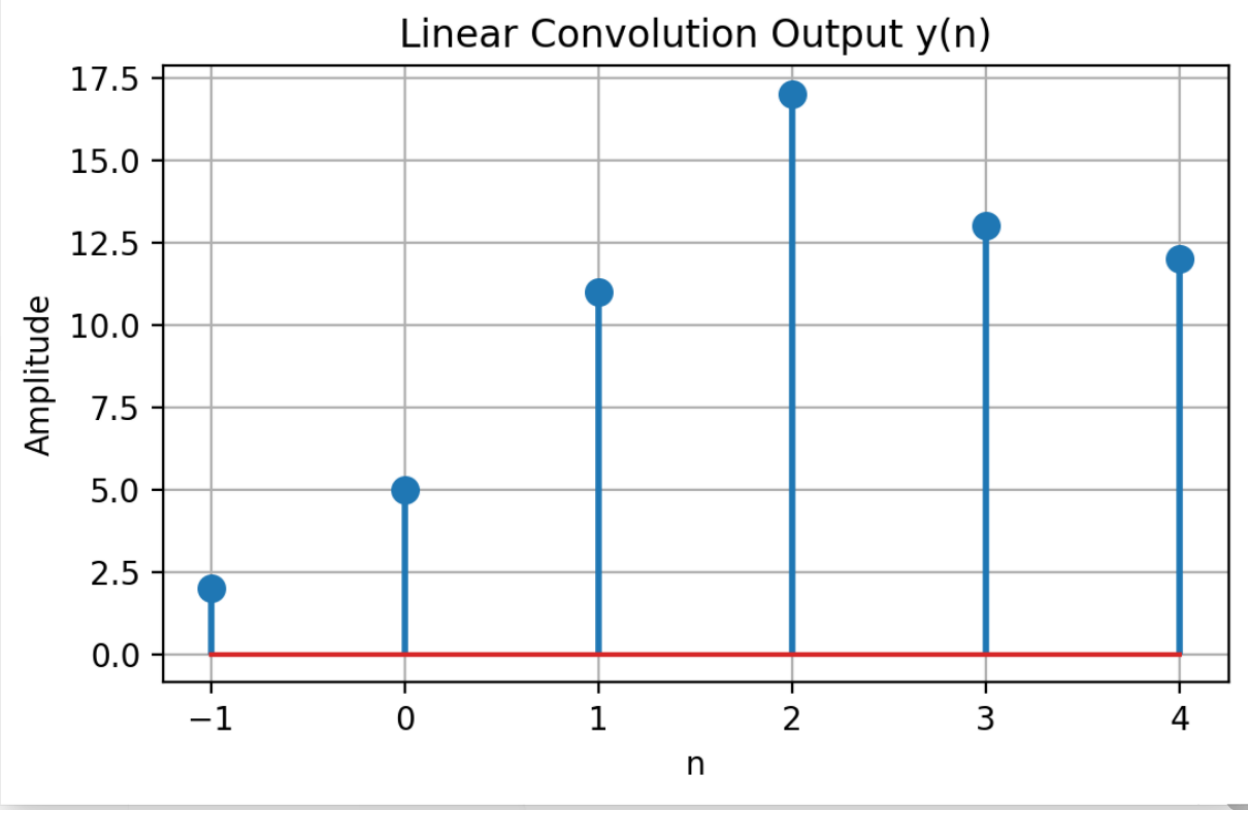
Sample Output:

```
Linear Convolution Output =

    2     5    11    17    13    12
```


Input Sequence x(n)
Amplitude
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
n


Input Sequence h(n)
Amplitude
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
n



Result

Thus, the MATLAB program to perform **linear convolution of two discrete-time sequences without using the built-in conv() function** was successfully executed. The input sequences and the resulting output sequence were plotted correctly

Experiment 2 a) : Linear Convolution

Aim

To write a MATLAB program to perform the linear convolution of two discrete-time sequences without using the built-in conv() function and to plot the input and output sequences.

Simulation Tools / Hardware Required

Software

MATLAB R2021a or later (any recent version)

Hardware

Personal Computer/Laptop

Theory

Linear convolution is a mathematical operation used to determine the output of a **Linear Time-Invariant (LTI)** system for a given input sequence.

If

$$x(n) = \{x_0, x_1, \ldots, x_{N-1}\}$$

and

$$h(n) = \{h_0, h_1, \ldots, h_{M-1}\}$$

then the linear convolution is

$$y(n) = x(n) * h(n)$$

where

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k)$$

The length of the output sequence is

$$L = N + M - 1$$

In this experiment, the convolution is performed manually using **nested for-loops**, without using MATLAB's built-in **conv()** function.

The starting index of both sequences may be any integer (positive, zero, or negative). Accordingly, the output sequence starting index is

$$n_y = n_x + n_h$$

where

= Starting index of input sequence , = Starting index of impulse response

Algorithm

1. Start the program.
2. Enter the first discrete sequence.
3. Enter its starting index.
4. Enter the second discrete sequence.
5. Enter its starting index.
6. Find the lengths of both sequences.
7. Initialize the output sequence with zeros.
8. Perform linear convolution using two nested loops.
9. Compute the output starting index.
10. Generate the corresponding index values.
11. Plot the first input sequence.
12. Plot the second input sequence.
13. Plot the output sequence.
14. Display the convolution result.
15. Stop.

Program

```
clc;
clear;
close all;

% Input first sequence
x = input('Enter the first sequence x(n): ');
sx = input('Enter the starting index of x(n): ');

% Input second sequence
h = input('Enter the second sequence h(n): ');
sh = input('Enter the starting index of h(n): ');

N = length(x);
M = length(h);

% Output length
L = N + M - 1;
```

```
% Initialize output
y = zeros(1,L);

% Linear convolution without built-in function
for i = 1:N
   for j = 1:M
      y(i+j-1) = y(i+j-1) + x(i)*h(j);
   end
end

% Index generation
nx = sx : sx+N-1;
nh = sh : sh+M-1;
sy = sx + sh;
ny = sy : sy+L-1;

disp('Linear Convolution Output = ');
disp(y);

% Plotting
figure;
subplot(3,1,1);
stem(nx,x,'filled','LineWidth',2);
grid on;
xlabel('n');
ylabel('Amplitude');
title('Input Sequence x(n)');

subplot(3,1,2);
stem(nh,h,'filled','LineWidth',2);
grid on;
xlabel('n');
ylabel('Amplitude');
title('Input Sequence h(n)');
```

```
subplot(3,1,3);
stem(ny,y,'filled','LineWidth',2);
grid on;
xlabel('n');
ylabel('Amplitude');
title('Linear Convolution Output y(n)');
```

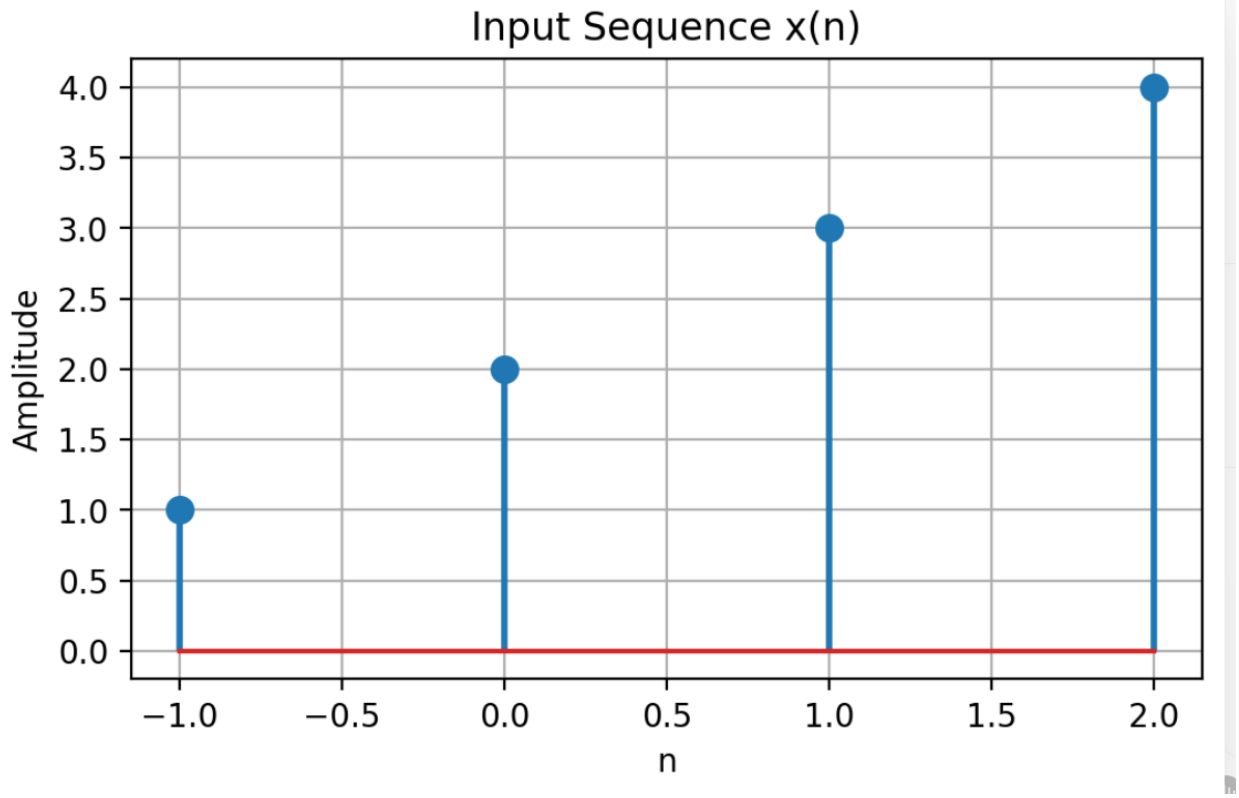
Sample Input

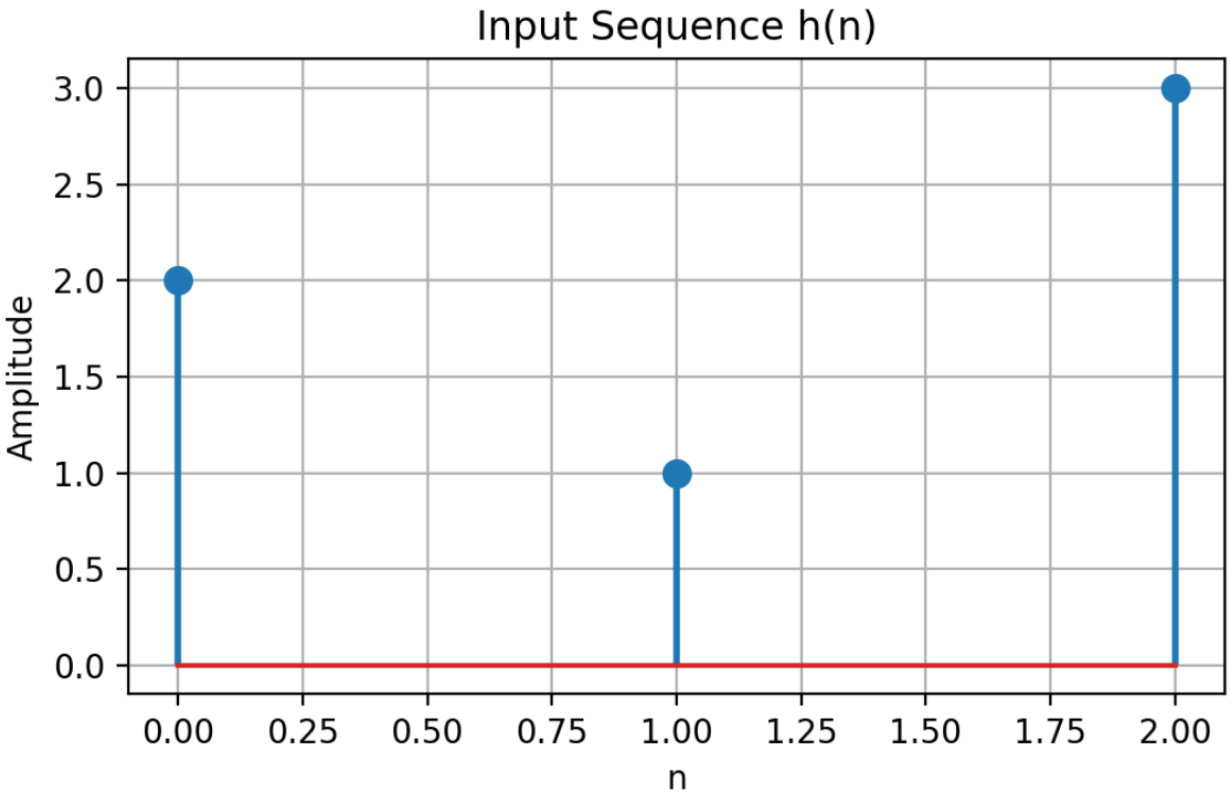
```
Enter the first sequence x(n):
[1 2 3 4]
Enter the starting index of x(n):
-1
Enter the second sequence h(n):
[2 1 3]
Enter the starting index of h(n):
0
```

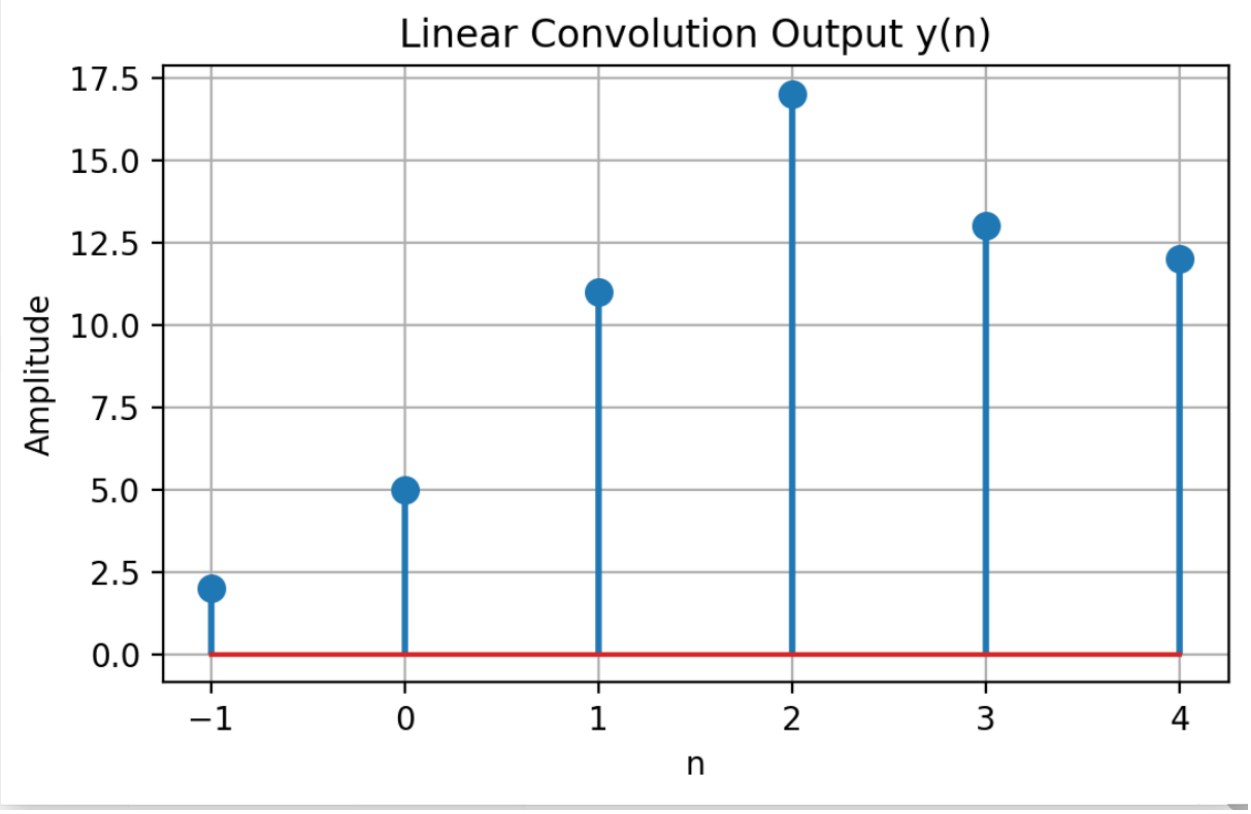
Sample Output:

```
Linear Convolution Output =

    2     5    11    17    13    12
```


Input Sequence x(n)
Amplitude
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
n


Input Sequence h(n)
Amplitude
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
n



Result

Thus, the MATLAB program to perform **linear convolution of two discrete-time sequences without using the built-in conv() function** was successfully executed. The input sequences and the resulting output sequence were plotted correctly

Experiment No 2b) Circular Convolution

AIM

To write a MATLAB program to perform **N-point circular convolution** between two discrete-time sequences **without using the built-in cconv() function** and display the input and output sequences using stem plots.

SOFTWARE / HARDWARE REQUIRED

MATLAB R2020a or later
Windows/Linux Operating System
Personal Computer with minimum 4 GB RAM

THEORY

Circular convolution is used extensively in Digital Signal Processing (DSP), especially in the implementation of filtering using the Discrete Fourier Transform (DFT).

For two finite-length sequences

$$x(n), \qquad h(n)$$

their **N-point circular convolution** is defined as

$$y(n) = \sum_{k=0}^{N-1} x(k)\, h((n-k) \bmod N)$$

where

- N is the length of the circular convolution.
- The modulo operation wraps the sequence around the circle whenever the index becomes negative or exceeds $N-1$.

Unlike linear convolution, the sequences are assumed to be periodic with period N.

Case 1: Both sequences have equal length

If

$$N_x = N_h = N$$

Circular convolution is performed directly.

Example

$$x(n) = \{1\ 2\ 3\ 4\}$$

$$h(n) = \{2\ 1\ 0\ 1\}$$

Length

$$N = 4$$

Case 2: Sequences have unequal lengths

If

$$N_x \neq N_h$$

the shorter sequence must be **zero padded** until both sequences become equal length.

Example

$$x(n) = \{1\ 2\ 3\}$$

$$h(n) = \{4\ 5\}$$

Pad the shorter sequence

$$h(n) = \{4\ 5\ 0\}$$

Now

$$N = 3$$

Circular convolution can then be performed.

ALGORITHM

1. Start.
2. Enter the first sequence.
3. Enter the second sequence.
4. Find the lengths of both sequences.
5. Determine the maximum length.
6. Zero-pad the shorter sequence.
7. Initialize the output sequence with zeros.
8. For each output sample
   - Multiply corresponding samples.
   - Use modulo indexing to wrap around the sequence.
   - Accumulate the products.
9. Display the circular convolution output.
10. Plot
    - First input sequence
    - Second input sequence
    - Circular convolution output
11. Stop.

MATLAB PROGRAM

```
clc;
clear;
close all;

% Read input sequences
x = input('Enter first sequence x(n): ');
h = input('Enter second sequence h(n): ');

% Lengths of sequences
Nx = length(x);
Nh = length(h);

% Equalize lengths by zero padding
N = max(Nx,Nh);

if Nx < N
   x = [x zeros(1,N-Nx)];
```

```
end

if Nh < N
   h = [h zeros(1,N-Nh)];
end

% Circular convolution without using cconv()

y = zeros(1,N);

for n = 1:N
   for k = 1:N

      index = mod((n-k),N);

      if index == 0
         index = N;
      end

      y(n) = y(n) + x(k)*h(index);

   end
end

% Display result

disp('First sequence')
disp(x)

disp('Second sequence')
disp(h)

disp('Circular convolution output')
disp(y)

% Plotting

figure;

subplot(3,1,1)
stem(0:N-1,x,'filled','LineWidth',2)
```

```
grid on
xlabel('n')
ylabel('x(n)')
title('Input Sequence x(n)')

subplot(3,1,2)
stem(0:N-1,h,'filled','LineWidth',2)
grid on
xlabel('n')
ylabel('h(n)')
title('Input Sequence h(n)')

subplot(3,1,3)
stem(0:N-1,y,'filled','LineWidth',2)
grid on
xlabel('n')
ylabel('y(n)')
title('Circular Convolution Output')
```

SAMPLE INPUT

Example 1 (Equal Length)

```
Enter first sequence x(n):
[1 2 3 4]

Enter second sequence h(n):
[2 1 0 1]
```

OUTPUT

```
First sequence

     1     2     3     4

Second sequence

     2     1     0     1

Circular convolution output

     12     12     16     20
```

Figure 1: Input Sequence $x(n)$

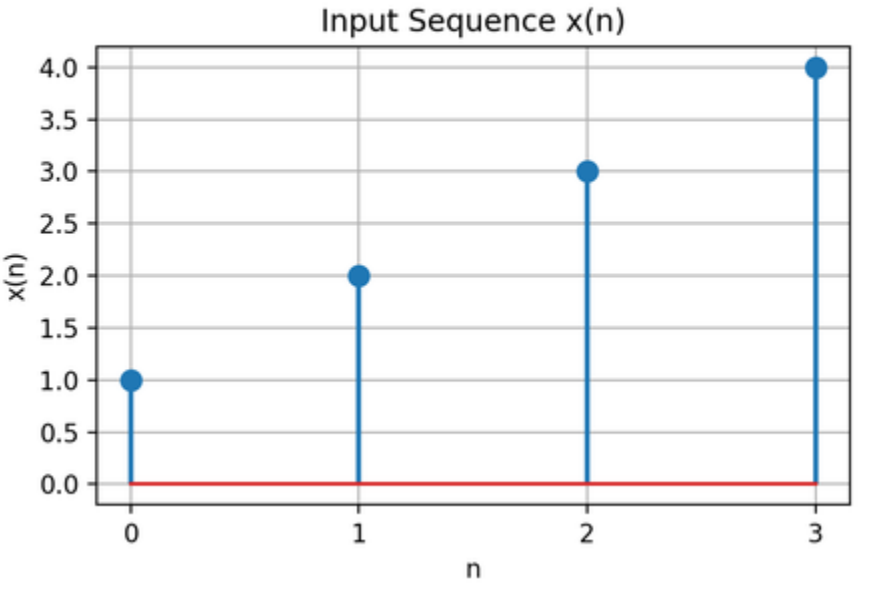


Figure 2: Input Sequence $h(n)$

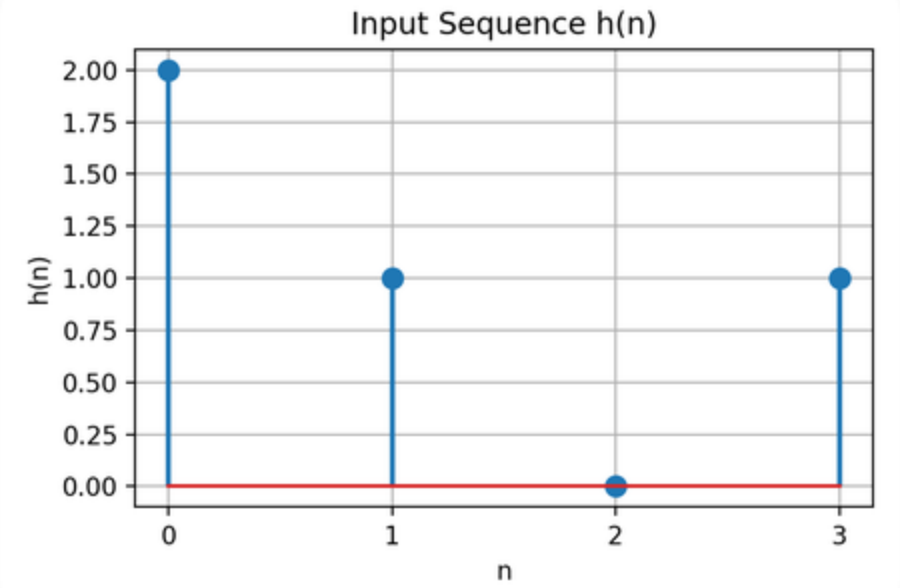
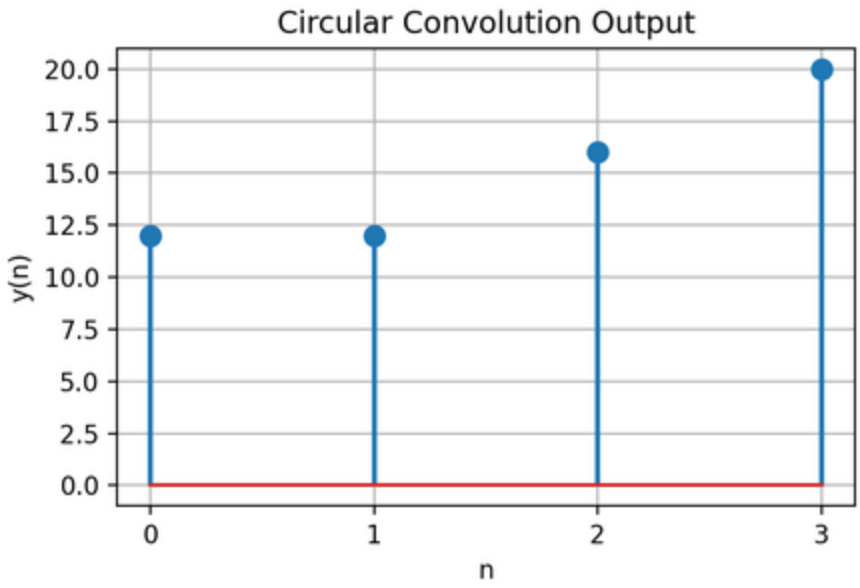


Figure 3: Circular Convolution Output



Example 2 (Unequal Length)

Input

```
x(n)=[1 2 3]

h(n)=[4 5]
```

After zero padding

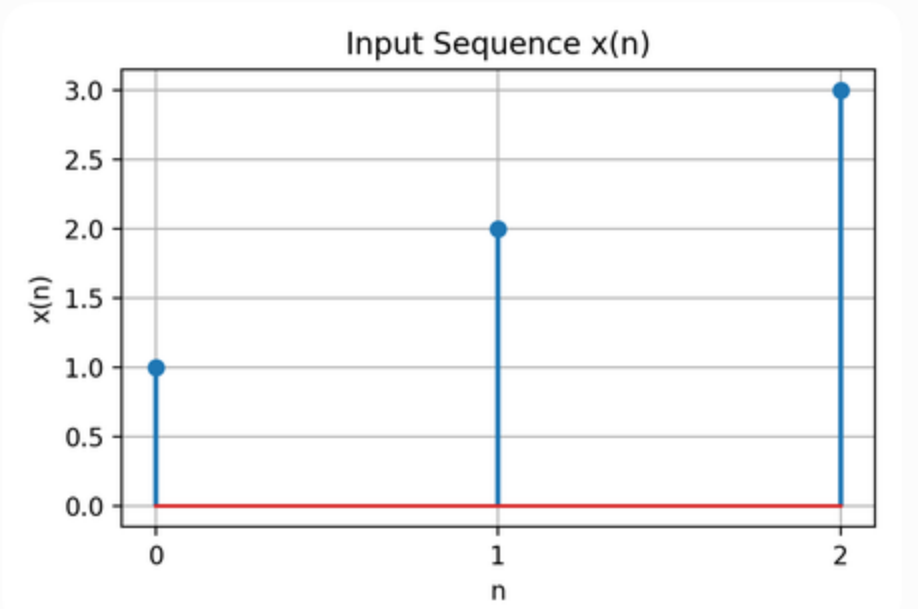
```
x=[1 2 3]

h=[4 5 0]
```

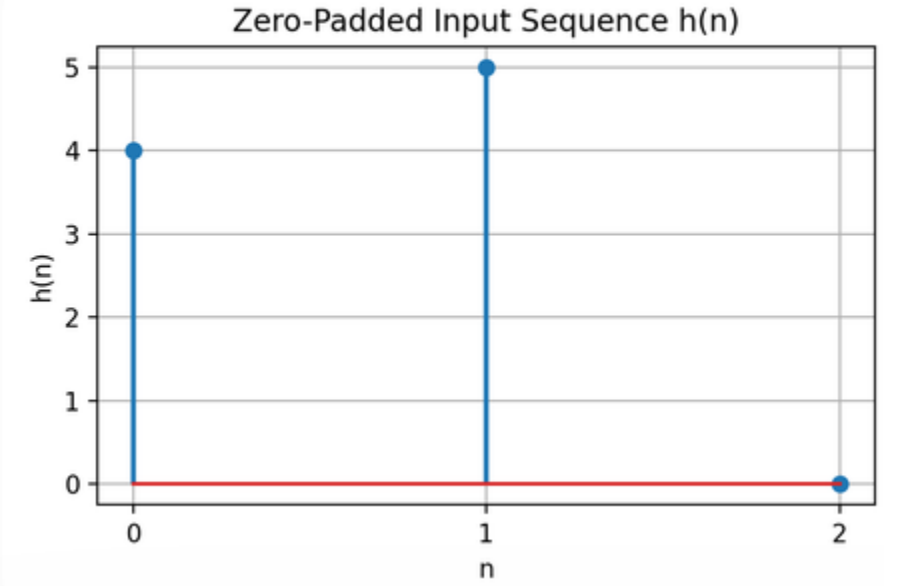
Output

```
y=[19 13 22]
```

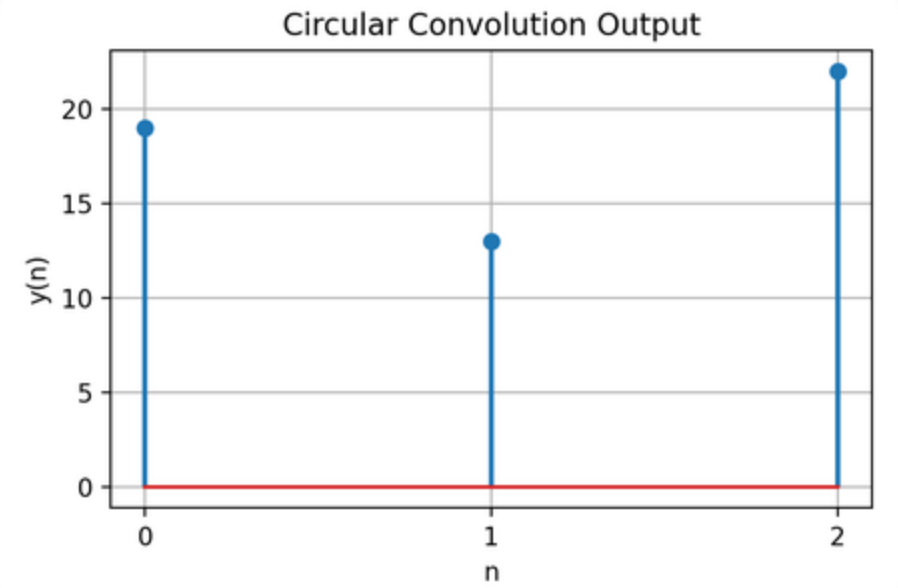
Input Sequence $x(n)$:



Zero-Padded Input Sequence $h(n)$:



Circular Convolution Output $y(n)$:



RESULT

Thus, the MATLAB program to perform **circular convolution** of two discrete-time sequences **without using the built-in cconv() function** was successfully executed, and the input and output sequences were displayed using stem plots.

Experiment No 2 c) Linear Convolution by means of Circular Convolution

Aim

To perform linear convolution of two discrete-time sequences using circular convolution without using MATLAB built-in functions conv() and cconv().

Hardware/Software Required

Personal Computer

MATLAB R2020a or later

Windows

Theory

For sequences of lengths N1 and N2, linear convolution length is L=N1+N2−1. Zero-pad both sequences to length L and perform L-point circular convolution using modulo indexing. The result equals linear convolution because zero-padding prevents time-domain aliasing.

THEORY

Linear convolution of two finite-length sequences is given by

$$y(n) = x(n) * h(n)$$

where

- $x(n)$ has length N_1
- $h(n)$ has length N_2

The length of the linear convolution is

$$L = N_1 + N_2 - 1$$

Linear convolution can also be obtained by performing **L-point circular convolution**, provided both sequences are zero-padded to length

$$L = N_1 + N_2 - 1$$

The circular convolution formula is

$$y(n) = \sum_{k=0}^{L-1} x(k)\, h((n-k) \bmod L)$$

Since both sequences are padded with zeros, no aliasing occurs, and the circular convolution result becomes exactly equal to the linear convolution.

Example

Given

$$x(n) = \{1, 2, 3\}$$

$$h(n) = \{4, 5\}$$

Lengths

$$N_1 = 3, \qquad N_2 = 2$$

Linear convolution length

$$L = 3 + 2 - 1 = 4$$

Zero-padded sequences

```
x=[1 2 3 0]

h=[4 5 0 0]
```

Performing **4-point circular convolution** gives

```
y=[4 13 22 15]
```

which is exactly the linear convolution.

ALGORITHM

1. Start.
2. Enter the first sequence.
3. Enter the second sequence.
4. Find the lengths of both sequences.
5. Compute

$$L = N_1 + N_2 - 1$$

6. Zero-pad both sequences to length **L**.
7. Initialize the output sequence with zeros.
8. Perform L-point circular convolution using nested loops and modulo indexing.
9. Display the output sequence.
10. Plot the two input sequences and output sequence.
11. Stop.

MATLAB PROGRAM

```
clc;
clear;
close all;

% Read input sequences

x = input('Enter first sequence x(n): ');
h = input('Enter second sequence h(n): ');

N1 = length(x);
N2 = length(h);

% Length required for linear convolution

L = N1 + N2 - 1;

% Zero padding

x = [x zeros(1,L-N1)];
h = [h zeros(1,L-N2)];

% Circular convolution

y = zeros(1,L);
```

```
for n = 1:L

   for k = 1:L

      index = mod((n-k),L);

      if index==0
         index=L;
      end

      y(n)=y(n)+x(k)*h(index);

   end

end

disp('Zero padded x(n)')
disp(x)

disp('Zero padded h(n)')
disp(h)

disp('Linear convolution using circular convolution')

disp(y)

% Plotting

figure

subplot(3,1,1)
stem(0:L-1,x,'filled','LineWidth',2)
grid on
xlabel('n')
ylabel('x(n)')
title('Zero-Padded Input Sequence x(n)')

subplot(3,1,2)
stem(0:L-1,h,'filled','LineWidth',2)
grid on
xlabel('n')
ylabel('h(n)')
title('Zero-Padded Input Sequence h(n)')
```

```
subplot(3,1,3)
stem(0:L-1,y,'filled','LineWidth',2)
grid on
xlabel('n')
ylabel('y(n)')
title('Linear Convolution using Circular Convolution')
```

SAMPLE INPUT

```
Enter first sequence

[1 2 3]

Enter second sequence

[4 5]
```

OUTPUT

```
Zero padded x(n)

1     2     3     0

Zero padded h(n)

4     5     0     0

Linear convolution using circular convolution

4    13    22    15
```

INPUT FIGURES

Input Sequence

```
x(n)=[1 2 3]
```

After zero padding

```
x(n)=[1 2 3 0]
```

Second Sequence

```
h(n)=[4 5]
```

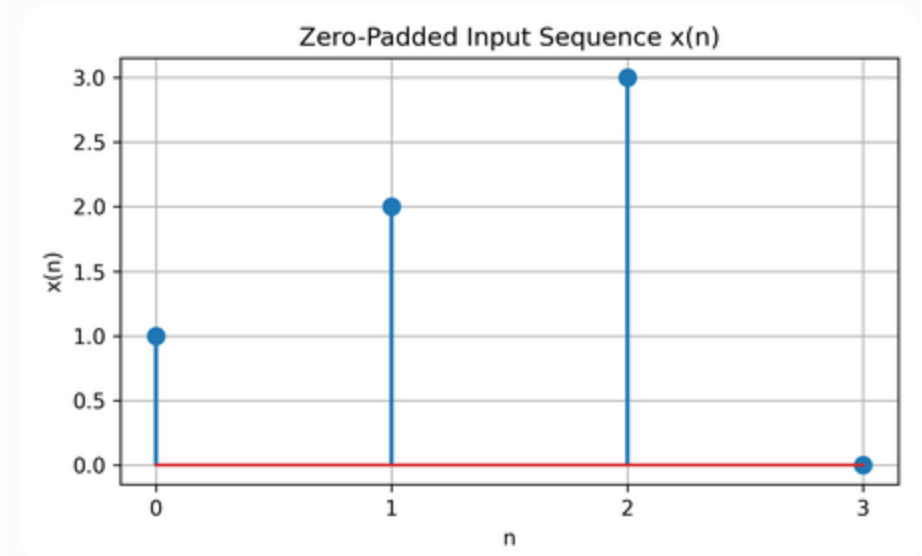
After zero padding

```
h(n)=[4 5 0 0]
```

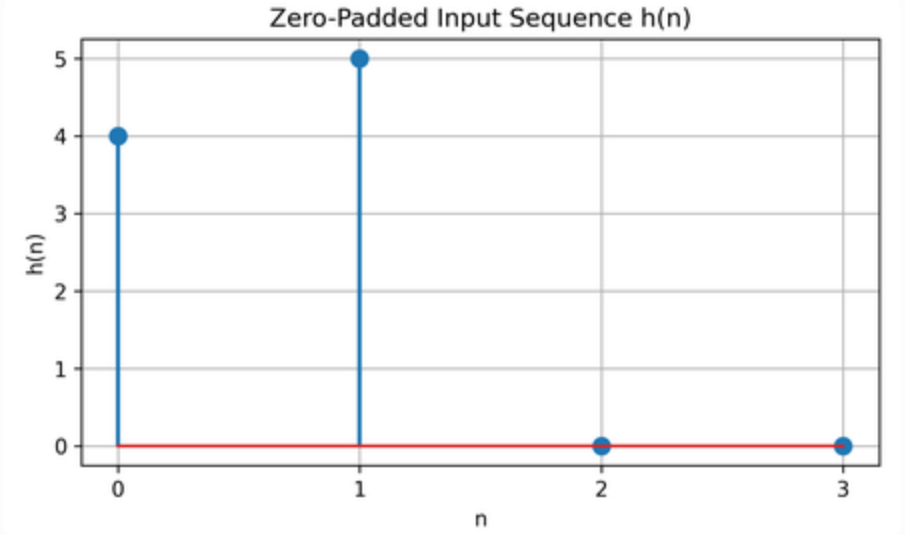
OUTPUT FIGURE

```
y(n)=[4 13 22 15]
```

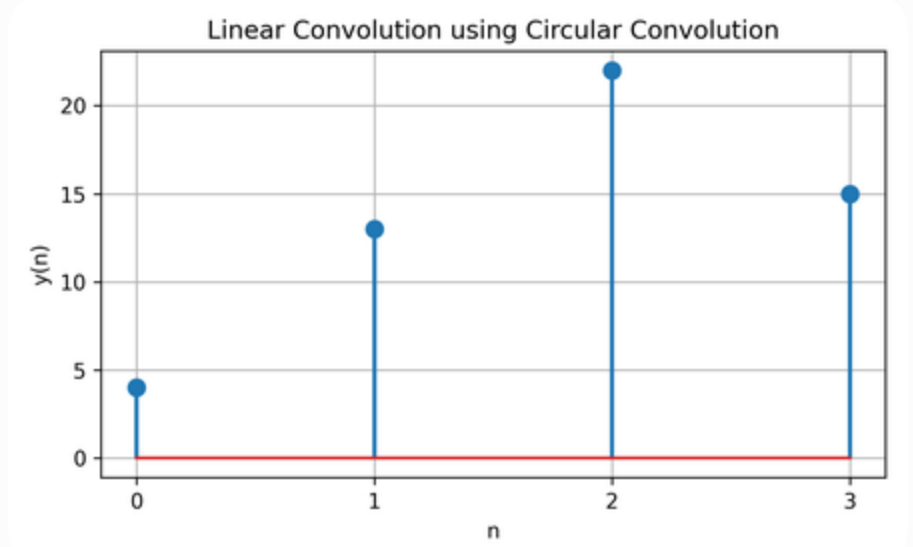
Zero-Padded Input Sequence $x(n)$



Zero-Padded Input Sequence $h(n)$



Linear Convolution Using Circular Convolution (Output)



RESULT

Thus, the MATLAB program to perform **linear convolution using circular convolution** without using the built-in functions **conv()** and **cconv()** was successfully executed, and the input and output sequences were displayed using stem plots.

Expt No 3 a)

AUTOCORRELATION OF A DISCRETE-TIME SIGNAL WITHOUT USING THE BUILT-IN FUNCTION

Aim

To write a MATLAB program to compute the **autocorrelation** of a discrete-time signal **without using the built-in xcorr() function** and to plot the input sequence and the autocorrelation sequence.

Software / Hardware Requirements

Hardware

Personal Computer (PC) or Laptop

Intel Core i3 Processor or above

Minimum 4 GB RAM

Software

MATLAB R2018a or above

Windows/Linux Operating System

Theory

Autocorrelation is a mathematical operation used to measure the similarity between a discrete-time signal and its delayed version. It indicates how closely a signal resembles itself at different time shifts (lags). Autocorrelation is widely used in digital signal processing for detecting periodicity, estimating signal energy, and analysing random signals.

The autocorrelation of a discrete-time signal is given by

$$R_{xx}(k) = \sum_{n=-\infty}^{\infty} x(n)x(n-k)$$

where

- $x(n)$ = Input sequence
- k = Lag (shift)
- $R_{xx}(k)$ = Autocorrelation sequence

For a finite-length sequence of length N,

- the lag varies from

$$-(N-1) \leq k \leq (N-1)$$

Hence, the length of the autocorrelation sequence is

$$2N-1$$

Properties of Autocorrelation

1. The autocorrelation sequence is **symmetric.**

$$R_{xx}(k) = R_{xx}(-k)$$

2. The maximum value occurs at **zero lag.**
3. The length of the autocorrelation sequence is

$$2N - 1$$

Algorithm

1. Start the program.
2. Read the input sequence .
3. Read the starting index of the input sequence.
4. Determine the length of the sequence.
5. Generate the time index for the input sequence.
6. Generate lag values from to .
7. Initialize the autocorrelation sequence to zero.
8. Use two nested **for** loops to compute the autocorrelation manually.
9. Store the computed values.
10. Display the autocorrelation sequence.
11. Plot the input sequence.
12. Plot the autocorrelation sequence.
13. Stop.

MATLAB Program

```
%-----------------------------------------------------------
% Program to Perform Autocorrelation without using xcorr()
%-----------------------------------------------------------

clc;
clear;
close all;

% Read the input sequence
x = input('Enter the input sequence x(n): ');

% Read the starting index
n_start = input('Enter the starting index of x(n): ');
```

```
% Length of input sequence
N = length(x);

% Generate time index
n = n_start:n_start+N-1;

% Initialize autocorrelation sequence
R = zeros(1,2*N-1);

% Generate lag values
lag = -(N-1):(N-1);

% Compute autocorrelation
for k = -(N-1):(N-1)

   sum1 = 0;

   for i = 1:N

      j = i-k;

      if(j>=1 && j<=N)

         sum1 = sum1 + x(i)*x(j);

      end

   end

   R(k+N)=sum1;

end

% Display results
disp('Input Time Index');
disp(n);

disp('Autocorrelation Sequence');
disp(R);
```

```
% Plot input sequence
figure;
stem(n,x,'filled','LineWidth',1.5);
grid on;
xlabel('Time Index (n)');
ylabel('Amplitude');
title('Input Sequence x(n)');

% Plot autocorrelation
figure;
stem(lag,R,'filled','LineWidth',1.5);
grid on;
xlabel('Lag (k)');
ylabel('R_{xx}(k)');
title('Autocorrelation Sequence');
```

Sample Input

```
Input Sequence

x(n)=[1 2 3 4]

Starting Index = 0
```

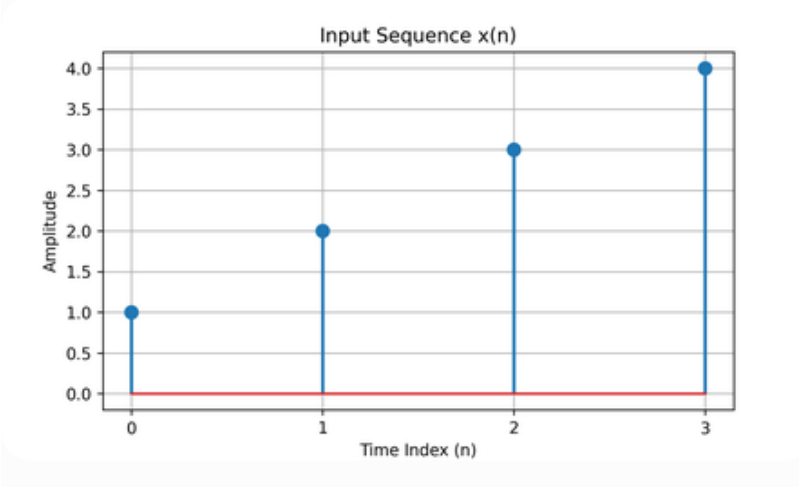
Sample Output

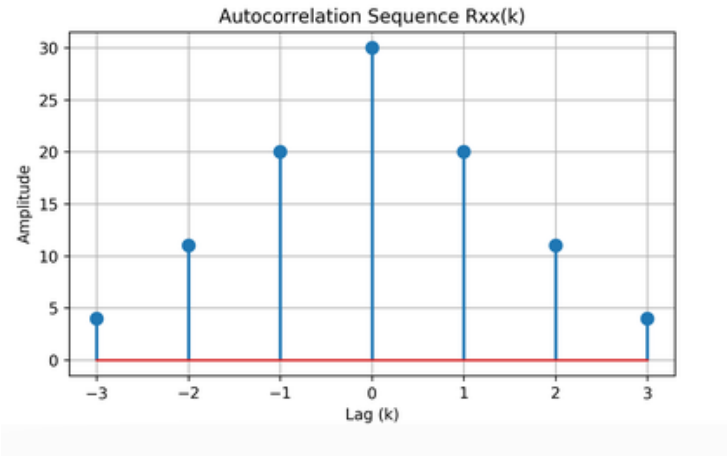
```
Input Time Index

0     1     2     3

Autocorrelation Sequence

4    11    20    30    20    11    4
```





Result

Thus, the autocorrelation of the given discrete-time sequence was computed without using the built-in xcorr() function using MATLAB, and the corresponding input sequence and autocorrelation sequence were plotted successfully.

Experiment No 3 b).

Cross-Correlation of Two Discrete-Time Sequences without using Built-

in Function.

AIM

To write a MATLAB program to compute the **cross-correlation** between two discrete-time sequences **without using the built-in xcorr() function** and to plot the input sequences and the resulting cross-correlation sequence.

SOFTWARE REQUIREMENTS

MATLAB R2020a or later (Any recent version)

Windows Operating System

HARDWARE REQUIREMENTS

Personal Computer (PC)

THEORY

Cross-correlation is used to measure the similarity between two discrete-time signals as one signal is shifted with respect to the other. It helps determine the degree of similarity between two sequences at different time shifts (lags).

Unlike convolution, cross-correlation does not reverse the first sequence. Instead, the second sequence is time-reversed before convolution.

Mathematically,

$$r_{xy}(l) = \sum_{n=-\infty}^{\infty} x(n)y(n-l)$$

where

- $x(n)$ = First sequence
- $y(n)$ = Second sequence
- l = Lag

Cross-correlation can also be expressed as

$$r_{xy}(l) = x(n) * y(-n)$$

where

- $y(-n)$ is the time-reversed version of $y(n)$
- $*$ denotes linear convolution.

The output sequence length is

$$L = L_x + L_y - 1$$

where

- L_x = Length of first sequence
- L_y = Length of second sequence

ALGORITHM

Step 1

Start the program.

Step 2

Read the first input sequence and its starting index.

Step 3

Read the second input sequence and its starting index.

Step 4

Find the lengths of both sequences.

Step 5

Reverse the second sequence manually.

Step 6

Determine the starting index of the reversed sequence.

Step 7

Perform linear convolution between and the reversed sequence using nested loops.

Step 8

Compute the index values of the output sequence.

Step 9

Display the cross-correlation sequence.

Step 10

Plot

Input sequence

Input sequence

Cross-correlation output

Step 11

Stop.

MATLAB PROGRAM

```
% Program to Perform Cross-Correlation without using   xcorr()
clc;
clear;
close all;

% Read the first sequence
x = input('Enter the first sequence x(n): ');
nx = input('Enter the starting index of x(n): ');

% Read the second sequence
y = input('Enter the second sequence y(n): ');
ny = input('Enter the starting index of y(n): ');

Lx = length(x);
Ly = length(y);

% Time Reverse the Second Sequence

yr = zeros(1,Ly);
```

```
for i = 1:Ly
   yr(i)=y(Ly-i+1);
end
nyr=-(ny+Ly-1);

% Linear Convolution

r=zeros(1,Lx+Ly-1);

for i=1:Lx
   for j=1:Ly
      r(i+j-1)=r(i+j-1)+x(i)*yr(j);
   end
end

% Output Index

nr=nx+nyr;
index=nr:nr+length(r)-1;
% Display Output

disp('Cross Correlation Sequence')
disp(r)
disp('Index Values')
disp(index)

% Plotting

figure
subplot(3,1,1)
stem(nx:nx+Lx-1,x,'filled','LineWidth',1.5)
grid on
xlabel('n')
ylabel('Amplitude')
title('Input Sequence x(n)')

subplot(3,1,2)
stem(ny:ny+Ly-1,y,'filled','LineWidth',1.5)
grid on
xlabel('n')
ylabel('Amplitude')
title('Input Sequence y(n)')
```

```
subplot(3,1,3)
stem(index,r,'filled','LineWidth',1.5)
grid on
xlabel('Lag')
ylabel('Amplitude')
title('Cross Correlation Output')
```

SAMPLE INPUT

```
Enter the first sequence x(n) : [1 2 3 4]

Starting Index :0

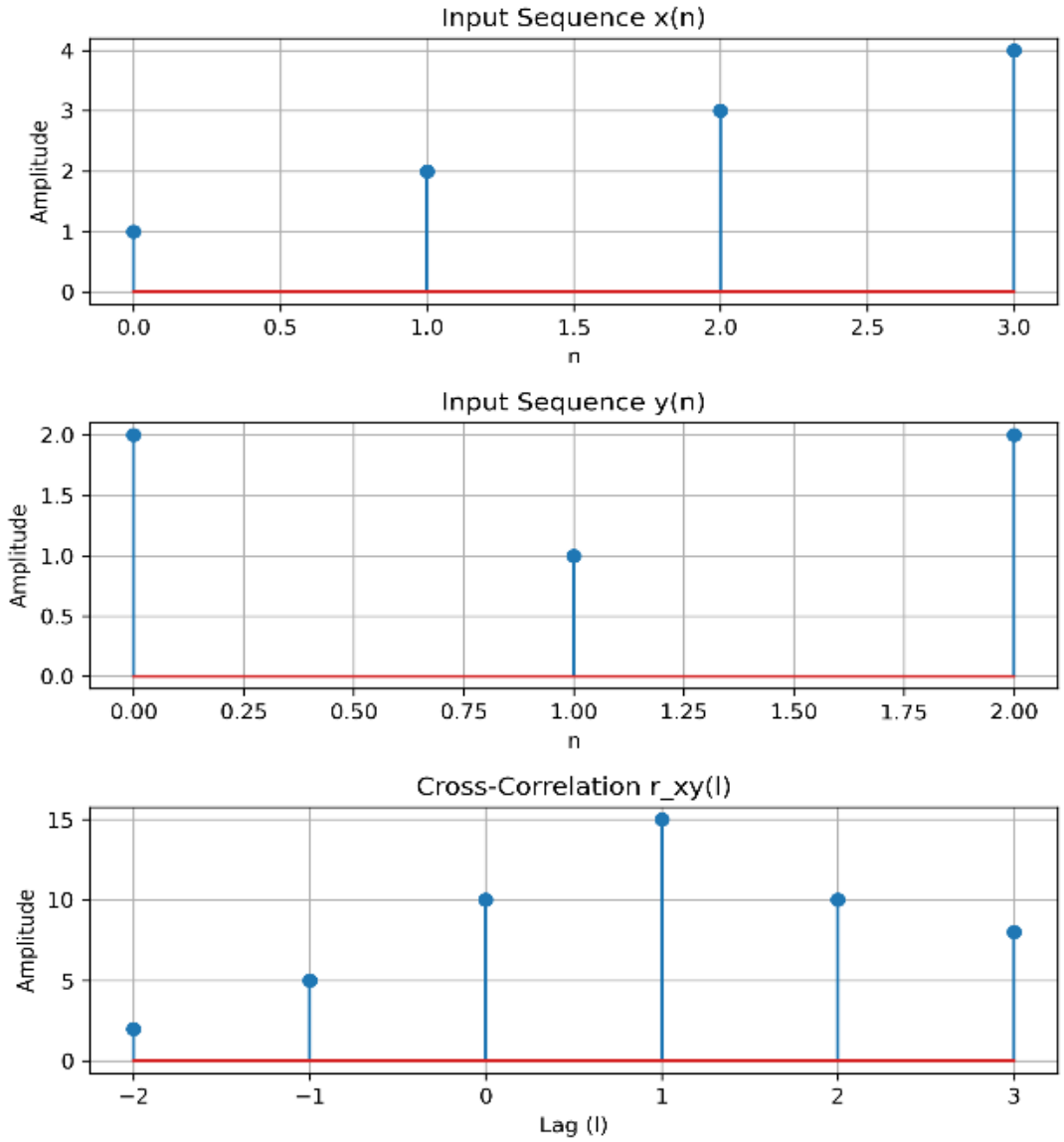
Enter the second sequence y(n):[2 1 2]

Starting Index:0
```

OUTPUT

```
Cross Correlation Sequence:    2    5    10    15    10    8

Index Values:   -2   -1    0    1    2    3
```



RESULT

Thus, the **cross-correlation** between two discrete-time sequences was successfully computed **without using the built-in xcorr() function**. The input sequences and the resulting cross-correlation sequence were plotted successfully using MATLAB.

Experiment No 4: Frequency analysis Using DFT

Aim

To write and execute a MATLAB program to compute the **Discrete Fourier Transform (DFT)** of a discrete-time signal and analyze its **frequency spectrum** by plotting the magnitude and phase spectra.

Hardware Requirements

Personal Computer (PC)

Software Requirements

MATLAB R2018a or later
Windows Operating System

Theory

The **Discrete Fourier Transform (DFT)** converts a finite-length discrete-time sequence into its corresponding frequency-domain representation. It provides information about the frequency components present in the signal.

For an input sequence $x(n)$ of length N, the DFT is defined as

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j\frac{2\pi}{N}kn}$$

where

- $x(n)$ = Input sequence
- $X(k)$ = DFT of the sequence
- N = Number of samples
- $k = 0, 1, 2, \ldots, N-1$

The DFT produces complex-valued coefficients. The frequency characteristics of the signal are represented using:

Magnitude Spectrum

$$|X(k)| = \sqrt{Re(X(k))^2 + Im(X(k))^2}$$

which indicates the strength (amplitude) of each frequency component.

Phase Spectrum

$$\angle X(k) = \tan^{-1}\left(\frac{Im(X(k))}{Re(X(k))}\right)$$

which indicates the phase shift of each frequency component.

Thus, frequency analysis is performed by plotting the magnitude and phase spectra of the DFT coefficients.

Algorithm

1. Start the program.
2. Enter the input discrete-time sequence $x(n)$.
3. Determine the length N of the sequence.
4. Initialize the DFT output sequence $X(k)$ to zero.
5. Compute the DFT using the equation

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}$$

   for $k = 0$ to $N - 1$.
6. Compute the magnitude spectrum using

$$|X(k)| = abs(X(k))$$

7. Compute the phase spectrum using

$$\angle X(k) = angle(X(k))$$

8. Display the DFT coefficients.
9. Plot the input sequence.
10. Plot the magnitude spectrum.
11. Plot the phase spectrum.
12. Stop.

MATLAB Program

```
%-------------------------------------------------------
% Program: Frequency Analysis using DFT
%-------------------------------------------------------
clc;
clear;
close all;

% Read the input sequence
x = input('Enter the input sequence x(n): ');

% Length of the sequence
N = length(x);

% Compute N-point DFT
X = zeros(1,N);
```

```
for k = 0:N-1
   sum = 0;
   for n = 0:N-1
      sum = sum + x(n+1)*exp(-1j*2*pi*k*n/N);
   end
   X(k+1) = sum;
end

% Magnitude and Phase Spectrum
Mag = abs(X);
Phase = angle(X);

% Frequency Axis
k = 0:N-1;

% Display Results
disp('Input Sequence x(n):');
disp(x);

disp('DFT Coefficients X(k):');
disp(X);

disp('Magnitude Spectrum:');
disp(Mag);

disp('Phase Spectrum (radians):');
disp(Phase);

% Plot Results
figure;

subplot(3,1,1);
stem(0:N-1,x,'filled');
xlabel('n');
ylabel('Amplitude');
title('Input Sequence');
grid on;

subplot(3,1,2);
stem(k,Mag,'filled');
xlabel('Frequency Index (k)');
ylabel('|X(k)|');
title('Magnitude Spectrum');
grid on;

subplot(3,1,3);
stem(k,Phase,'filled');
xlabel('Frequency Index (k)');
ylabel('\angle X(k) (radians)');
title('Phase Spectrum');
grid on;
```

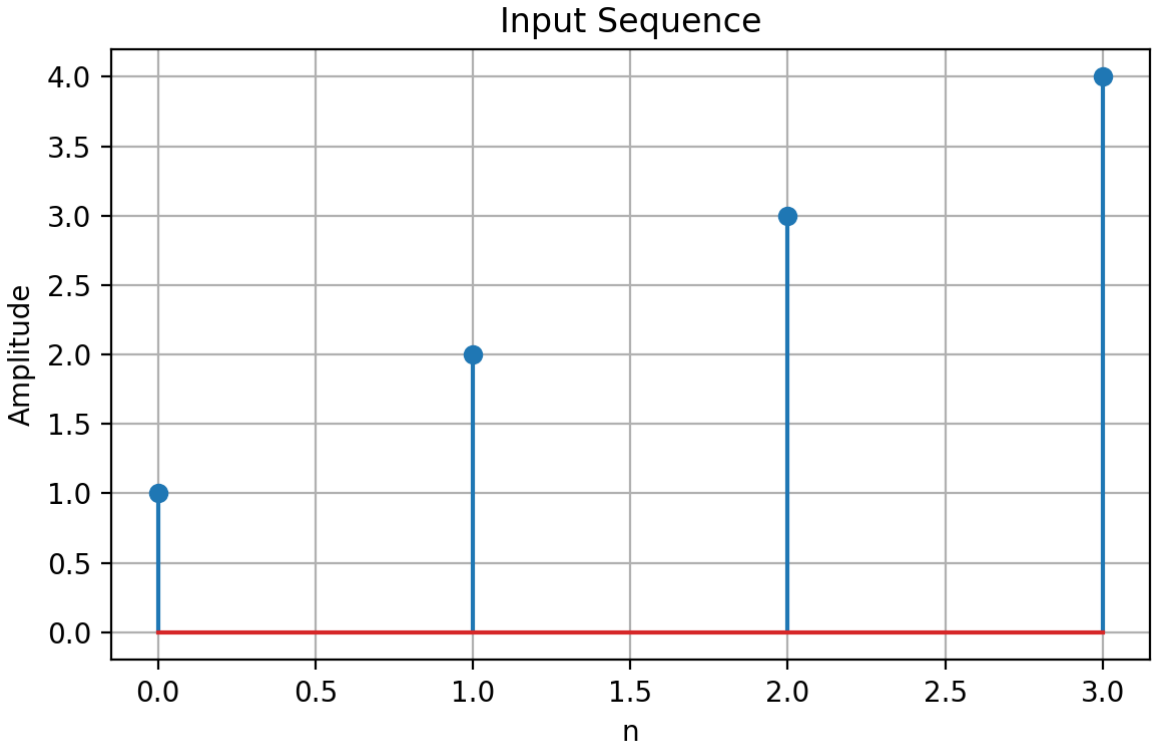
Sample Output

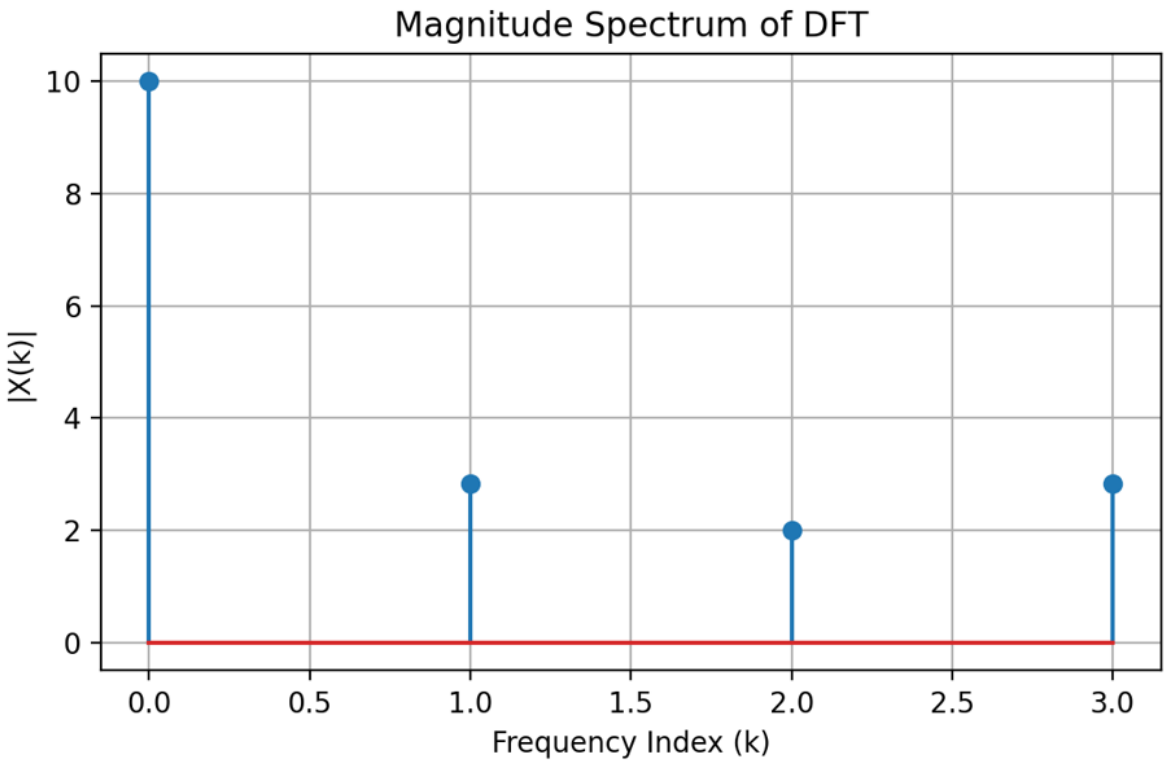
Input Sequence x(n):
1 2 3 4

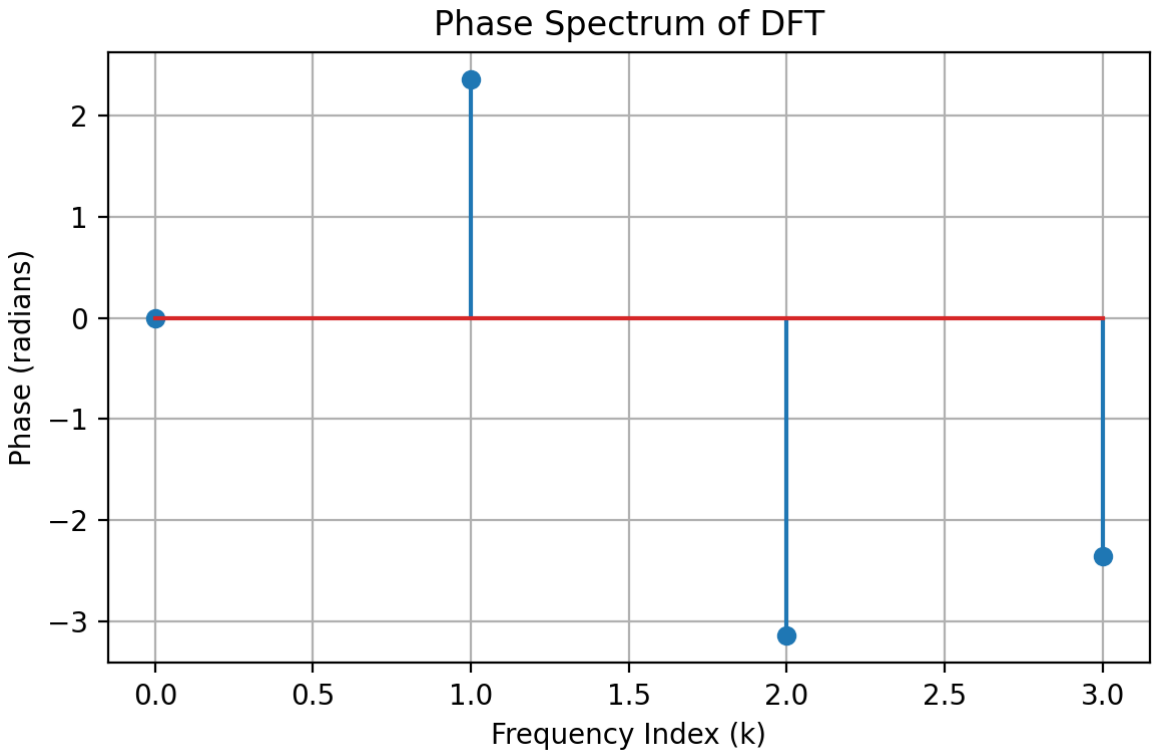
DFT Coefficients X(k):
10.0000 + 0.0000i
-2.0000 + 2.0000i
-2.0000 + 0.0000i
-2.0000 - 2.0000i

Magnitude Spectrum:
10.0000
2.8284
2.0000
2.8284

Phase Spectrum (radians):
0
2.3562
3.1416
-2.3562







Result

Thus, the MATLAB program to perform **frequency analysis using the Discrete Fourier Transform (DFT)** was successfully executed. The DFT coefficients, magnitude

spectrum, and phase spectrum of the given discrete-time sequence were obtained and plotted successfully.

Experiment No 5.

Design of FIR Low Pass, High Pass, Band Pass and Band Reject Filters using Window Method (Without using fir1() and hamming ())

AIM

To design Finite Impulse Response (FIR) Low Pass Filter (LPF), High Pass Filter (HPF), Band Pass Filter (BPF), and Band Reject Filter (BRF) using the Window Method without using the MATLAB built-in functions fir1() and hamming(), and to plot the Impulse Response, Magnitude Response, and Phase Response.

SIMULATION TOOLS REQUIRED

- MATLAB R2020a or later
- Windows Operating System

HARDWARE REQUIRED

- Personal Computer (PC) / Laptop
- Intel Core i3 Processor or higher

THEORY

- A Finite Impulse Response (FIR) filter is a digital filter whose impulse response settles to zero in a finite duration. FIR filters are inherently stable and possess a linear phase response, making them suitable for many digital signal processing applications.
- The ideal impulse response of an FIR filter is obtained using the inverse Fourier transform. Since the ideal response is infinitely long, it must be truncated to obtain a realizable filter. This truncation introduces ripples in the frequency response known as the Gibbs phenomenon. To reduce these ripples, a window function such as the Hamming Window is applied.
- In this experiment, the Hamming window is generated manually without using the MATLAB built-in function hamming(). Similarly, the ideal FIR coefficients are computed directly without using the fir1() function.

Types of FIR Filters:

Low Pass Filter (LPF):

- Passes low-frequency components.
- Rejects high-frequency components.

High Pass Filter (HPF):

- Passes high-frequency components.
- Rejects low-frequency components.

Band Pass Filter (BPF):

- Passes frequencies within a specified range.
- Rejects frequencies outside the specified band.

Band Reject Filter (BRF):

- Rejects frequencies within a specified range.
- Passes frequencies outside the rejected band.

DESIGN EQUATIONS

Ideal Low Pass Filter

$$h(n) = \begin{cases} \frac{\omega_c}{\pi}, & n = 0 \\ \frac{\sin(\omega_c n)}{\pi n}, & n \neq 0 \end{cases}$$

Ideal High Pass Filter

$$h(n) = \begin{cases} 1 - \frac{\omega_c}{\pi}, & n = 0 \\ -\frac{\sin(\omega_c n)}{\pi n}, & n \neq 0 \end{cases}$$

Ideal Band Reject Filter

$$h(n) = \begin{cases} 1 - \frac{\omega_2 - \omega_1}{\pi}, & n = 0 \\ \frac{\sin(\omega_1 n) - \sin(\omega_2 n)}{\pi n}, & n \neq 0 \end{cases}$$

Hamming Window

$$w(n) = 0.54 - 0.46\cos\left(\frac{2\pi n}{N}\right)$$

Final FIR Coefficients

$$h_w(n) = h(n) \times w(n)$$

ALGORITHM

1. Start the program.
2. Enter the filter order N.
3. Enter the sampling frequency F_s.
4. Select the required FIR filter (LPF, HPF, BPF or BRF).
5. Enter the cutoff frequency (or lower and upper cutoff frequencies).
6. Compute the ideal impulse response using the corresponding FIR equation.
7. Generate the Hamming window manually.
8. Multiply the ideal impulse response with the Hamming window.
9. Compute the frequency response using FFT.
10. Plot the impulse response.
11. Plot the magnitude response.
12. Plot the phase response.
13. Stop.

MATLAB PROGRAM:

```
% Design of FIR Low Pass, High Pass, Band Pass and Band Reject Filters
% using Window Method (Without using fir1() and hamming())
% AIM:
% To design FIR filters manually and plot their impulse response,
% magnitude response and phase response.

clc;              % Clears the command window
clear;            % Removes all variables from workspace
close all;        % Closes all figure windows


%%%% INPUT SECTION
```

```
N = input('Enter Filter Order (Even Number) = '); % Read filter order from user (Example: 50)

Fs = input('Enter Sampling Frequency (Hz) = '); % Read sampling frequency (Example: 1000 Hz)

disp(' ');
disp('******** FIR FILTER DESIGN ********');
disp('1. Low Pass Filter');
disp('2. High Pass Filter');
disp('3. Band Pass Filter');
disp('4. Band Reject Filter');

ch = input('Enter your Choice = '); % Read filter type

%%%% CREATE SAMPLE INDEX
M = N/2; % Middle point of the filter

n = -M:M;
% Sample positions
% Example:
% N=20
% n = -10,-9,-8,...0,...8,9,10

%%%%%%%%%%%%%%%%% CALCULATE IDEAL IMPULSE RESPONSE

switch ch

% LOW PASS FILTER

case 1

  Fc = input('Enter Cutoff Frequency (Hz) = ');

  wc = 2*pi*Fc/Fs;
  % Convert analog frequency into digital frequency

  for k = 1:length(n)

    if n(k)==0

      h(k)=wc/pi;
      % Centre coefficient

    else

      h(k)=sin(wc*n(k))/(pi*n(k));
      % Sinc function

    end

  end
```

```
   title_name='FIR LOW PASS FILTER';

% HIGH PASS FILTER
case 2

   Fc=input('Enter Cutoff Frequency (Hz)=');

   wc=2*pi*Fc/Fs;

   for k=1:length(n)

      if n(k)==0

         h(k)=1-wc/pi;

      else

         h(k)=-sin(wc*n(k))/(pi*n(k));

      end

   end

   title_name='FIR HIGH PASS FILTER';

% BAND PASS FILTER
%%%
```

```
   title_name='FIR BAND PASS FILTER';

% BAND REJECT FILTER
case 4

   F1=input('Enter Lower Cutoff Frequency=');
   F2=input('Enter Upper Cutoff Frequency=');

   w1=2*pi*F1/Fs;
   w2=2*pi*F2/Fs;

   for k=1:length(n)

      if n(k)==0

         h(k)=1-(w2-w1)/pi;

      else

         h(k)=(sin(w1*n(k))-sin(w2*n(k)))/(pi*n(k));

      end

   end

   title_name='FIR BAND REJECT FILTER';

otherwise

   disp('Invalid Choice');
   return;

end

%%%%% HAMMING WINDOW

for k=1:N+1

   w(k)=0.54-0.46*cos((2*pi*(k-1))/N);

end

% Apply Hamming Window

h=h.*w;

%%%REQUENCY RESPONSE

L=1024; % Number of FFT points

H=fft(h,L); % Compute frequency response
```

```
f=(0:L-1)*Fs/L; % Frequency axis

%%%%% IMPULSE RESPONSE

figure

stem(n,h,'filled','LineWidth',1.5)

grid on

xlabel('n')

ylabel('h(n)')

title(['Impulse Response of ',title_name])

%%%%% MAGNITUDE RESPONSE
figure

plot(f,abs(H),'b','LineWidth',2)

grid on

xlabel('Frequency (Hz)')

ylabel('Magnitude')

title(['Magnitude Response of ',title_name])

%%%%%%% PHASE RESPONSE
figure

plot(f,angle(H),'r','LineWidth',2)

grid on

xlabel('Frequency (Hz)')

ylabel('Phase (Radians)')

title(['Phase Response of ',title_name])

%%%%%%%%%%%%%%%% END OF PROGRAM
```

SAMPLE INPUTS

Case 1 – Low Pass Filter

Parameter	Value
Filter Order	50
Sampling	1000

Frequency	Hz
Choice	1
Cutoff Frequency	200 Hz

Case 2 – High Pass Filter

Parameter	Value
Filter Order	50
Sampling Frequency	1000 Hz
Choice	2
Cutoff Frequency	200 Hz

Case 3 – Band Pass Filter

Parameter	Value
Filter Order	50
Sampling Frequency	1000 Hz
Choice	3
Lower Cutoff Frequency	200 Hz
Upper Cutoff Frequency	400 Hz

Case 4 – Band Reject Filter

Parameter	Value
Filter Order	50
Sampling Frequency	1000 Hz
Choice	4
Lower Cutoff Frequency	200 Hz
Upper Cutoff Frequency	400 Hz

OUTPUT
Low Pass Filter
Impulse Response

- Symmetric sinc waveform multiplied by the Hamming window.

Magnitude Response

- Passes frequencies from 0–200 Hz.
- Attenuates frequencies above 200 Hz.

Phase Response

- Approximately linear phase response.

High Pass Filter

Impulse Response

- Symmetric impulse response with alternating positive and negative coefficients.

Magnitude Response

- Rejects frequencies below 200 Hz.
- Passes frequencies above 200 Hz.

Phase Response

- Nearly linear.

Band Pass Filter

Impulse Response

- Oscillatory symmetric waveform.

Magnitude Response

- Passes frequencies between 200 Hz and 400 Hz.
- Rejects frequencies outside this band.

Phase Response

- Linear phase characteristic.

Band Reject Filter

Impulse Response

- Symmetric waveform with notch characteristics.

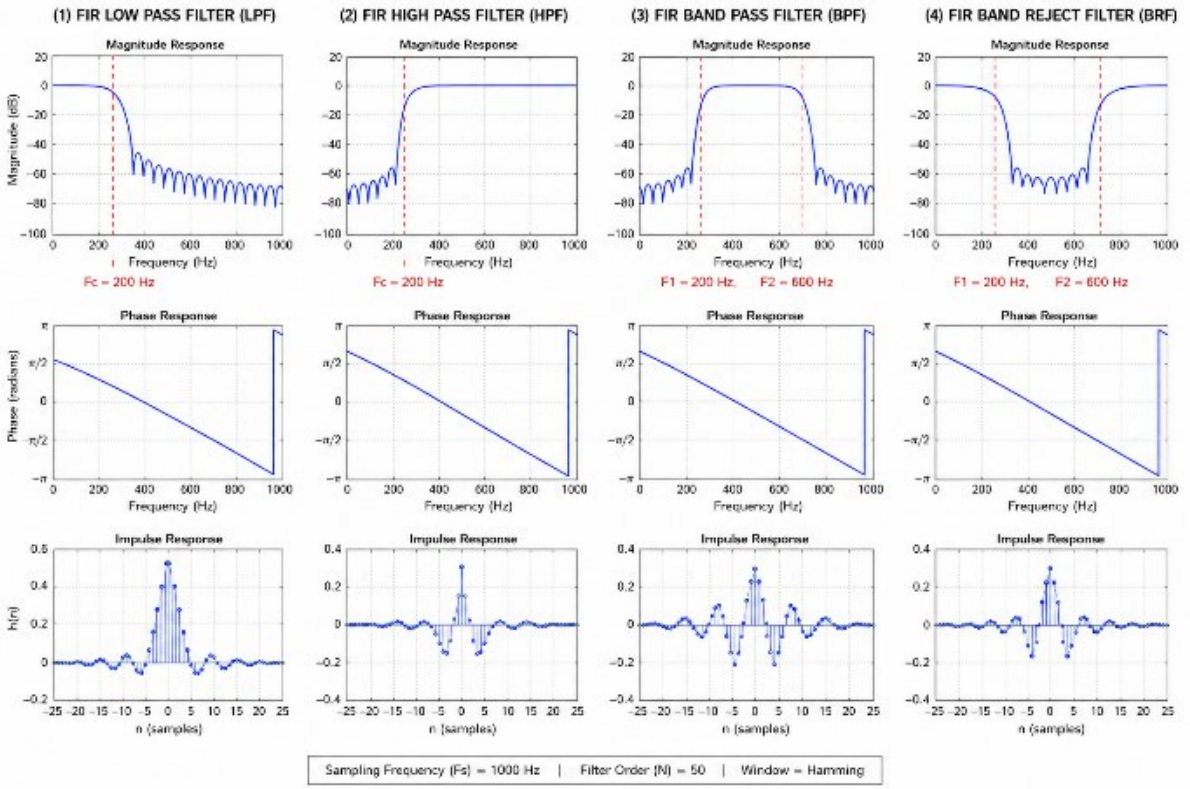
Magnitude Response

- Rejects frequencies between 200 Hz and 400 Hz.
- Passes frequencies below 200 Hz and above 400 Hz.

Phase Response

- Nearly linear.

Outputs



RESULT

The Finite Impulse Response (FIR) Low Pass Filter, High Pass Filter, Band Pass Filter, and Band Reject Filter were successfully designed using the Window Method without using the MATLAB built-in functions fir1() and hamming(). The impulse response, magnitude response, and phase response of each filter were obtained successfully, verifying the desired filtering characteristics. This confirms the correct implementation of manual FIR filter design using the Hamming window.

EX.NO: 6	DESIGN OF BUTTERWORTH AND CHEBYSHEV IIR FILTERS (LPF/HPF/BPF/BSF) AND DEMONSTRATES THE FILTERING OPERATIONS
DATE:	

AIM

To design Infinite Impulse Response (IIR) Low Pass Filter (LPF), High Pass Filter (HPF), Band Pass Filter (BPF), and Band Stop Filter (BSF) using the Butterworth and Chebyshev Type-I approximations, to plot their Magnitude Response, Phase Response and Pole-Zero locations, and to demonstrate the filtering operation of each designed filter on a composite test signal.

SIMULATION TOOLS REQUIRED

- MATLAB R2020a or later
- Windows Operating System

HARDWARE REQUIRED

- Personal Computer (PC) / Laptop
- Intel Core i3 Processor or higher

THEORY

An Infinite Impulse Response (IIR) filter is a digital filter that uses feedback, so its impulse response theoretically continues forever. IIR filters are described by a difference equation involving both past inputs and past outputs, which allows them to achieve a desired frequency response with a much lower order than an equivalent FIR filter.

Unlike FIR filters, IIR filters are generally not linear phase and can be unstable if the poles of the transfer function lie outside the unit circle. However, their computational efficiency makes them attractive for many practical applications.

- Butterworth Filter: Provides a magnitude response that is maximally flat in the passband, with no ripples, and rolls off monotonically towards the stopband. The squared magnitude response is given by $|H(j\Omega)|^2 = 1 / [1 + (\Omega/\Omega c)^{(2N)}]$, where N is the filter order. A higher order gives a sharper transition but requires more computation.
- Chebyshev Type-I Filter: Provides a sharper transition (roll-off) than a Butterworth filter of the same order by allowing an equiripple (equal ripple) variation in the passband, while the stopband response remains monotonic. The passband ripple is

specified in dB (Rp) and controls the trade-off between passband flatness and transition sharpness.

- For an identical set of specifications (passband/stopband edge frequencies, passband ripple, and stopband attenuation), the Chebyshev filter meets the specification with a lower order than the Butterworth filter, at the cost of ripples in the passband.

Types of IIR Filters:

- Low Pass Filter (LPF): Passes low-frequency components and attenuates high-frequency components above the cutoff frequency.
- High Pass Filter (HPF): Passes high-frequency components and attenuates low-frequency components below the cutoff frequency.
- Band Pass Filter (BPF): Passes frequencies that lie within a specified band and attenuates frequencies outside that band.
- Band Stop Filter (BSF): Attenuates (rejects) frequencies that lie within a specified band and passes frequencies outside that band.

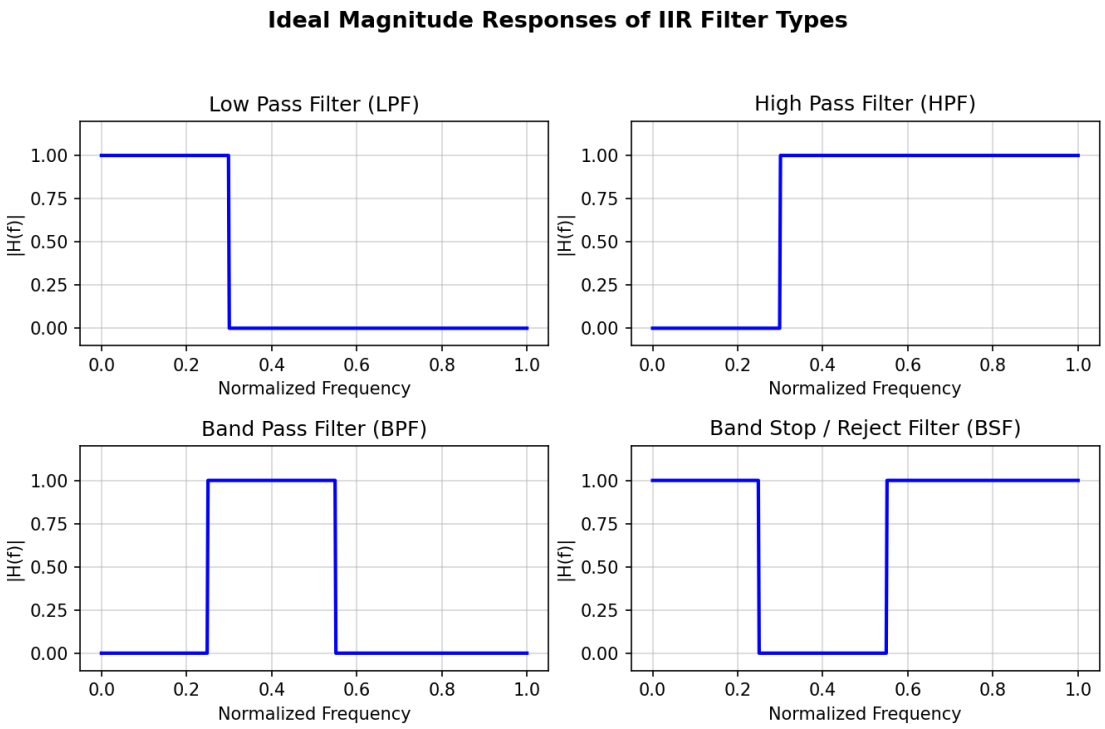


Fig. 1 Ideal magnitude responses of LPF, HPF, BPF and BSF

The plot below illustrates the fundamental difference between the two approximations for the same design specification: the Butterworth response is smooth and monotonic in the passband, whereas the Chebyshev response shows equiripple behaviour but achieves the required stopband attenuation with a lower filter order.

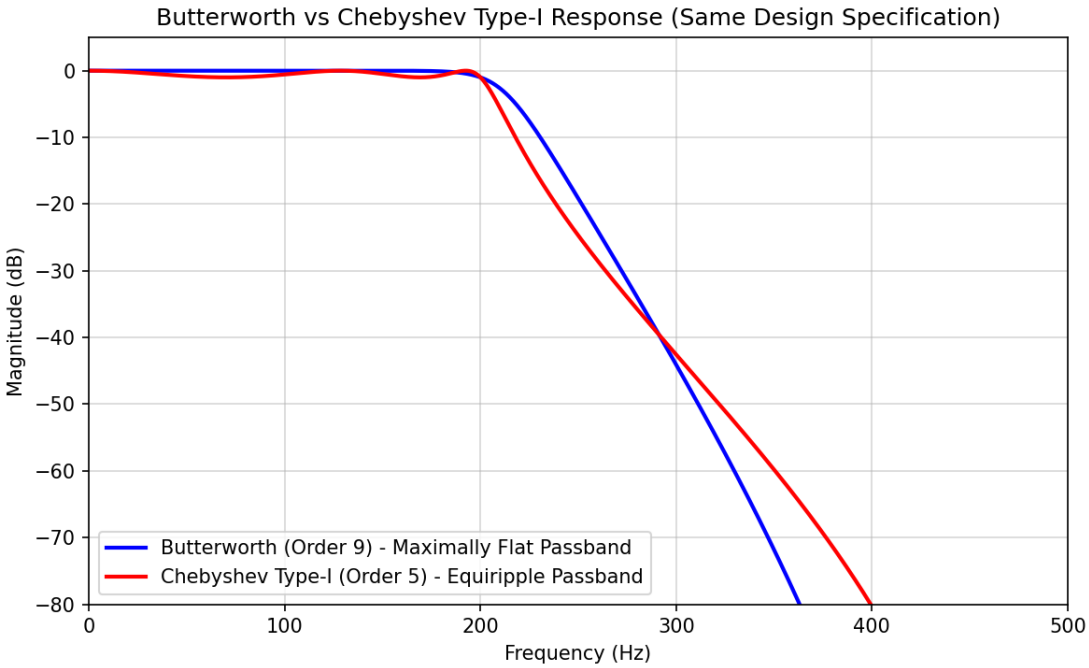


Fig. 2 Butterworth vs Chebyshev Type-I magnitude response for identical design specifications

MATLAB PROGRAM

```
clc;              % Clears the command window
clear;            % Removes all variables from workspace
close all;        % Closes all figure windows

%%%%% INPUT SECTION
Fs = input('Enter Sampling Frequency (Hz) = ');
disp(' ');
disp('******** SELECT FILTER TYPE ********');
disp('1. Low Pass Filter');
disp('2. High Pass Filter');
disp('3. Band Pass Filter');
disp('4. Band Stop Filter');
ch = input('Enter your Choice = ');      % Read filter type

disp(' ');
disp('******** SELECT FILTER CLASS ********');
disp('1. Butterworth');
disp('2. Chebyshev Type-I');
cls = input('Enter your Choice = ');     % Read filter class

Rp = 1;           % Passband ripple (dB) - used by Chebyshev
Rs = 40;          % Stopband attenuation (dB)
```

```
%%%%%%%%%%%%%%%%%% FREQUENCY SPECIFICATION, ORDER AND COEFFICIENTS
switch ch

% LOW PASS FILTER
case 1
   Fp = input('Enter Passband Frequency (Hz) = ');
   Fst = input('Enter Stopband Frequency (Hz) = ');
   Wp = Fp/(Fs/2);  Ws = Fst/(Fs/2);
   if cls == 1
      [N,Wn] = buttord(Wp,Ws,Rp,Rs);
      [b,a] = butter(N,Wn,'low');
   else
      [N,Wn] = cheb1ord(Wp,Ws,Rp,Rs);
      [b,a] = cheby1(N,Rp,Wn,'low');
   end
   title_name = 'IIR LOW PASS FILTER';

% HIGH PASS FILTER
case 2
   Fp = input('Enter Passband Frequency (Hz) = ');
   Fst = input('Enter Stopband Frequency (Hz) = ');
   Wp = Fp/(Fs/2);  Ws = Fst/(Fs/2);
   if cls == 1
      [N,Wn] = buttord(Wp,Ws,Rp,Rs);
      [b,a] = butter(N,Wn,'high');
   else
      [N,Wn] = cheb1ord(Wp,Ws,Rp,Rs);
      [b,a] = cheby1(N,Rp,Wn,'high');
   end
   title_name = 'IIR HIGH PASS FILTER';

% BAND PASS FILTER
case 3
   F1 = input('Enter Lower Cutoff Frequency (Hz) = ');
   F2 = input('Enter Upper Cutoff Frequency (Hz) = ');
   Wp = [F1 F2]/(Fs/2);  Ws = [F1-50 F2+50]/(Fs/2);
   if cls == 1
      [N,Wn] = buttord(Wp,Ws,Rp,Rs);
      [b,a] = butter(N,Wn,'bandpass');
   else
      [N,Wn] = cheb1ord(Wp,Ws,Rp,Rs);
```

```
      [b,a] = cheby1(N,Rp,Wn,'bandpass');
   end
   title_name = 'IIR BAND PASS FILTER';

% BAND STOP FILTER
case 4
   F1 = input('Enter Lower Cutoff Frequency (Hz) = ');
   F2 = input('Enter Upper Cutoff Frequency (Hz) = ');
   Wp = [F1 F2]/(Fs/2);  Ws = [F1+50 F2-50]/(Fs/2);
   if cls == 1
      [N,Wn] = buttord(Wp,Ws,Rp,Rs);
      [b,a] = butter(N,Wn,'stop');
   else
      [N,Wn] = cheb1ord(Wp,Ws,Rp,Rs);
      [b,a] = cheby1(N,Rp,Wn,'stop');
   end
   title_name = 'IIR BAND STOP FILTER';

otherwise
   disp('Invalid Choice');
   return;
end

%%%%%% FREQUENCY RESPONSE
[H,f] = freqz(b,a,1024,Fs);     % Compute frequency response

%%%%%% MAGNITUDE RESPONSE
figure
plot(f,20*log10(abs(H)),'b','LineWidth',2)
grid on
xlabel('Frequency (Hz)')
ylabel('Magnitude (dB)')
title(['Magnitude Response of ',title_name])

%%%%%%%% PHASE RESPONSE
figure
plot(f,unwrap(angle(H)),'r','LineWidth',2)
grid on
xlabel('Frequency (Hz)')
ylabel('Phase (Radians)')
title(['Phase Response of ',title_name])
```

```
%%%%%%%% POLE-ZERO PLOT
figure
zplane(b,a)
title(['Pole-Zero Plot of ',title_name])

%%%%%%%%%%%%%%%%%% DEMONSTRATION OF FILTERING OPERATION
t = 0:1/Fs:0.2;
% Composite test signal: 50 Hz + 250 Hz + 400 Hz + noise
x = sin(2*pi*50*t) + sin(2*pi*250*t) + sin(2*pi*400*t) + 0.3*randn(size(t));

y = filtfilt(b,a,x);      % Zero-phase filtering of the test signal

figure
subplot(2,1,1)
plot(t,x,'k')
grid on
xlabel('Time (s)'); ylabel('Amplitude')
title('Original (Noisy) Input Signal')

subplot(2,1,2)
plot(t,y,'b','LineWidth',1.5)
grid on
xlabel('Time (s)'); ylabel('Amplitude')
title(['Filtered Output Signal using ',title_name])

%%%%%%%%%%%%%%%%%% END OF PROGRAM
```

SAMPLE INPUTS

Case 1 & 2 - Low Pass Filter (Butterworth / Chebyshev)

Parameter	Value
Sampling Frequency (Fs)	1000 Hz
Filter Type Choice	1 (Low Pass)
Filter Class Choice	1 - Butterworth / 2 - Chebyshev
Passband Frequency (Fp)	200 Hz
Stopband Frequency (Fst)	300 Hz
Passband Ripple (Rp)	1 dB
Stopband Attenuation (Rs)	40 dB

Case 3 & 4 - High Pass Filter (Butterworth / Chebyshev)

Parameter	Value
Sampling Frequency (Fs)	1000 Hz
Filter Type Choice	2 (High Pass)
Filter Class Choice	1 - Butterworth / 2 - Chebyshev
Passband Frequency (Fp)	300 Hz
Stopband Frequency (Fst)	200 Hz
Passband Ripple (Rp)	1 dB
Stopband Attenuation (Rs)	40 dB

Case 5 & 6 - Band Pass Filter (Butterworth / Chebyshev)

Parameter	Value
Sampling Frequency (Fs)	1000 Hz
Filter Type Choice	3 (Band Pass)
Filter Class Choice	1 - Butterworth / 2 - Chebyshev
Lower Cutoff Frequency (F1)	150 Hz
Upper Cutoff Frequency (F2)	300 Hz
Passband Ripple (Rp)	1 dB
Stopband Attenuation (Rs)	40 dB

Case 7 & 8 - Band Stop Filter (Butterworth / Chebyshev)

Parameter	Value
Sampling Frequency (Fs)	1000 Hz
Filter Type Choice	4 (Band Stop)
Filter Class Choice	1 - Butterworth / 2 - Chebyshev
Lower Cutoff Frequency (F1)	150 Hz
Upper Cutoff Frequency (F2)	300 Hz

Passband Ripple (Rp)	1 dB
Stopband Attenuation (Rs)	40 dB

OUTPUT

Low Pass Filter

- Magnitude Response: Butterworth shows a smooth, monotonically decreasing response with no ripples in the passband (0-200 Hz), whereas Chebyshev shows a sharper roll-off with small equiripple variations in the passband.
- Phase Response: Both responses are nonlinear, with the Chebyshev filter showing a steeper phase change near the cutoff due to its sharper transition.

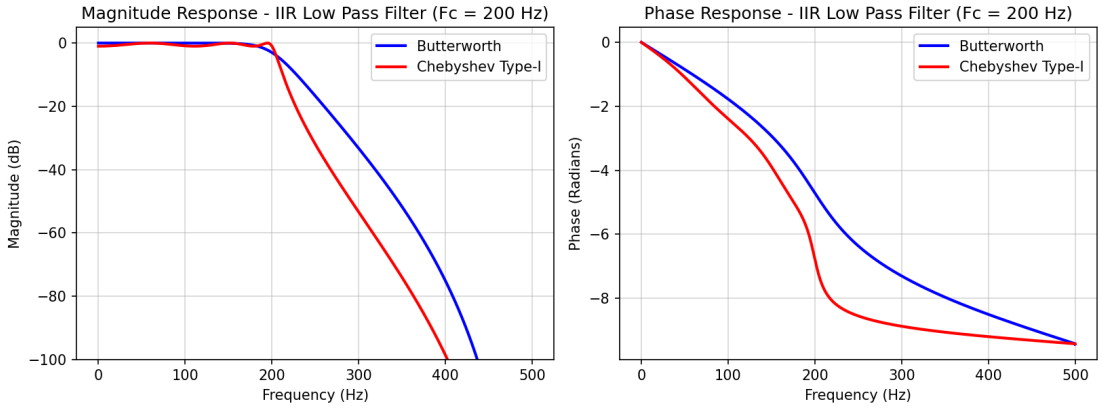


Fig. 3 Magnitude and phase response of Butterworth and Chebyshev IIR LPF

High Pass Filter

- Magnitude Response: Butterworth attenuates frequencies below 300 Hz smoothly; Chebyshev achieves the required attenuation with a sharper transition and passband ripple above 300 Hz.
- Phase Response: Nonlinear for both; the Chebyshev filter again exhibits a steeper phase transition near the cutoff frequency.

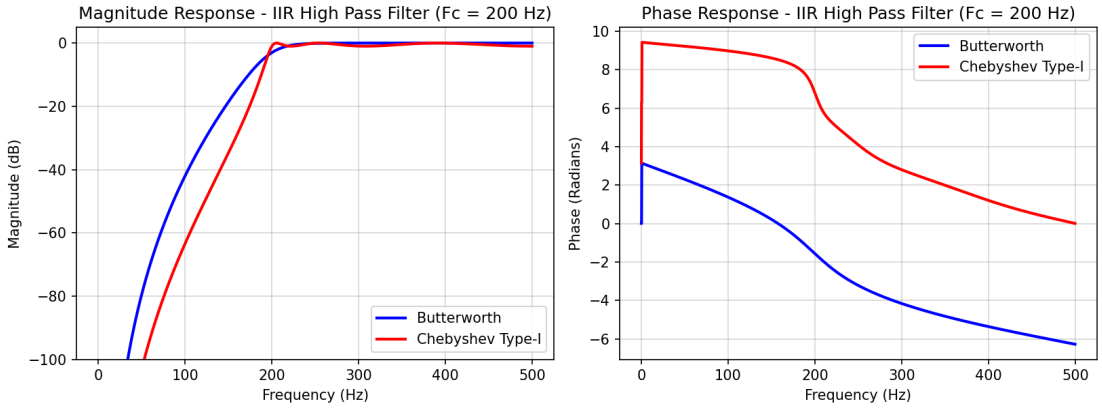


Fig. 4 Magnitude and phase response of Butterworth and Chebyshev IIR HPF

Band Pass Filter

- Magnitude Response: Both filters pass the 150-300 Hz band and attenuate frequencies outside it; the Chebyshev filter shows a narrower, sharper transition band for the same order.
- Phase Response: Rapid phase variation occurs within the passband for both filter classes.

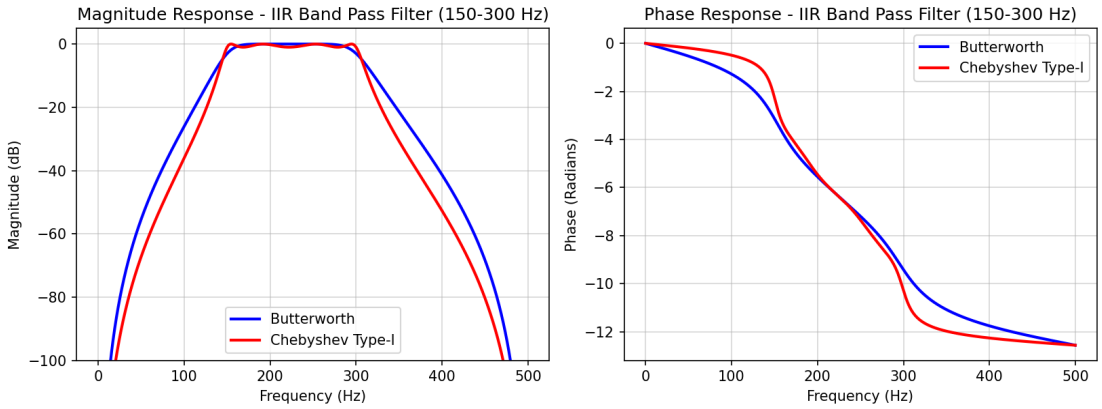


Fig. 5 Magnitude and phase response of Butterworth and Chebyshev IIR BPF

Band Stop Filter

- Magnitude Response: Both filters reject the 150-300 Hz band while passing frequencies outside it; Chebyshev again shows a sharper notch for the same filter order.
- Phase Response: A sharp phase discontinuity is observed near the rejected band for both filter classes.

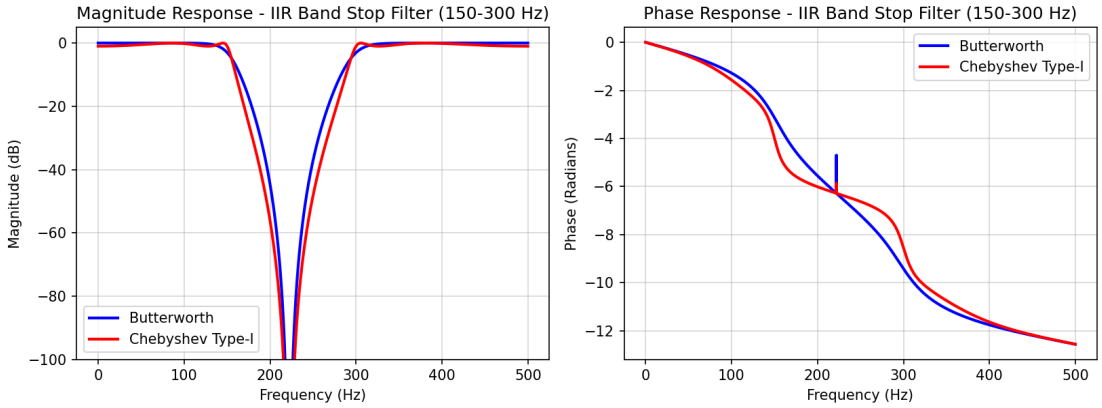


Fig. 6 Magnitude and phase response of Butterworth and Chebyshev IIR BSF

Pole-Zero Plot

The Butterworth filter poles are distributed uniformly on a circle inside the unit circle, giving the maximally flat response. The Chebyshev filter poles lie on an ellipse closer to the unit circle, giving a sharper roll-off at the cost of passband ripple. All poles for both designs lie within the unit circle, confirming that the filters are stable.

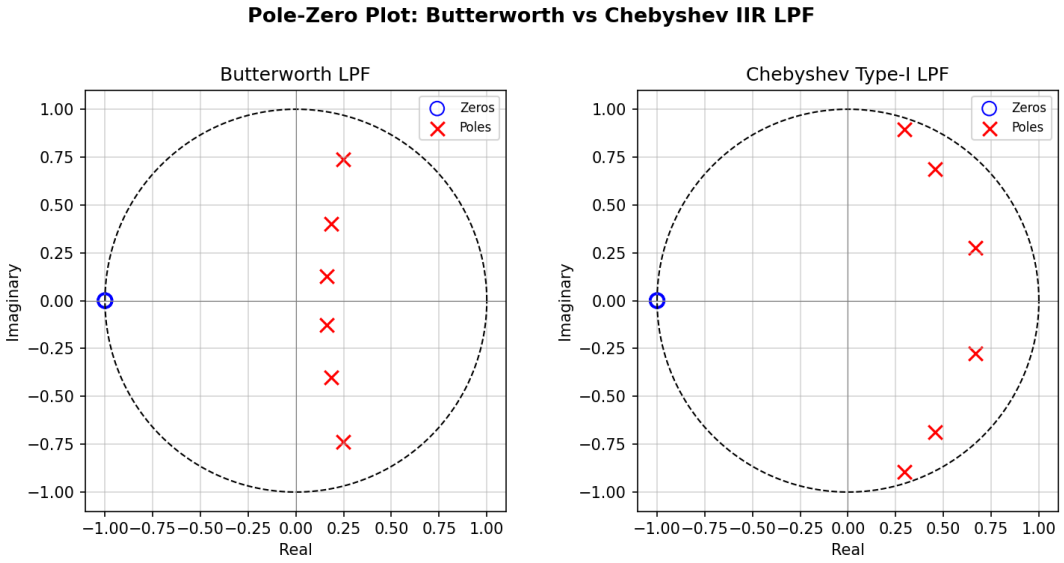


Fig. 7 Pole-zero plot of Butterworth and Chebyshev IIR LPF

Demonstration of Filtering Operation

- A composite test signal containing 50 Hz, 250 Hz and 400 Hz sinusoidal components corrupted with random noise was passed through each of the four designed filters using filtfilt() for zero-phase filtering.
- Low Pass Filter output: retains only the 50 Hz component and suppresses the 250 Hz and 400 Hz components along with most of the noise.
- High Pass Filter output: suppresses the 50 Hz component while retaining the 250 Hz and 400 Hz components.

- Band Pass Filter output: retains only the 250 Hz component, which lies within the 150-300 Hz passband, while rejecting the 50 Hz and 400 Hz components.
- Band Stop Filter output: rejects the 250 Hz component, which lies within the 150-300 Hz stopband, while retaining the 50 Hz and 400 Hz components.
- In every case, the Butterworth and Chebyshev outputs are close to each other but not identical - the Chebyshev output shows a slightly sharper separation of the desired and undesired components due to its steeper transition band, confirming the theoretical difference between the two filter classes.

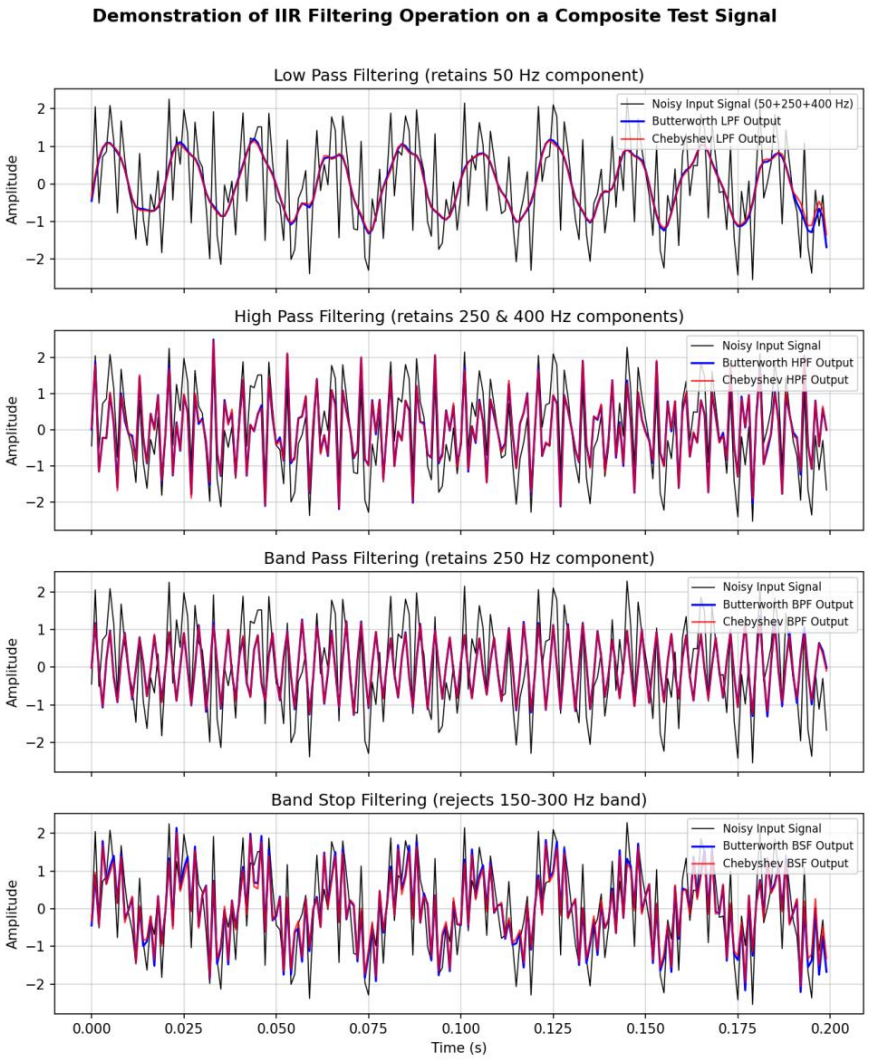


Fig. 8 Demonstration of LPF, HPF, BPF and BSF filtering operation on a composite test signal

RESULT

The Infinite Impulse Response (IIR) Low Pass Filter, High Pass Filter, Band Pass Filter and Band Stop Filter were successfully designed using both the Butterworth and Chebyshev Type-I approximations in MATLAB. The magnitude response, phase response and pole-zero locations of each filter were obtained and compared, verifying the maximally-flat characteristic of the Butterworth filter and the equiripple characteristic of the Chebyshev filter. This confirms the correct implementation and functioning of IIR filter design and filtering using MATLAB.

Expt. No. 7
Date 22.9.2026

Page No. 111

DEMONSTRATION OF FIR FILTER FOR LOW PASS, HIGH PASS, BAND PASS AND BAND STOP FILTER

AIM:

To design, implement and demonstrate Finite Impulse Response (FIR) Filters namely Low Pass Filter (LPF), High Pass Filter (HPF), Band Pass Filter (BPF), Band Stop Filter (BSF) using the TMS320C6713 DSP starter kit (DSK) and verify their frequency responses using Code Composer Studio (CCS).

HARDWARE REQUIREMENTS:

=> PC / Laptop
=> TMS320C6713 DSP starter kit
=> Connecting Cables

SOFTWARE REQUIREMENTS:

=> MATLAB R2020a or later
=> Code Composer Studio (3.1)
=> Siopefir
=> Topview DSP Debugger

THEORY:

A Finite Impulse Response (FIR) filter is one of the most important digital filters used in Digital Signal Processing (DSP). It is called a finite impulse response filter because its

response to a unit impulse input lasts for only a finite number of samples and eventually becomes zero. An FIR filter processes discrete-time signals by multiplying the current and previous input samples with a set of predetermined coefficients and then summing the results to produce the output. The coefficients determine the frequency response of the filter and are designed according to the desired filtering operation, such as low-pass, high-pass, band-pass or band stop filtering.

The mathematical equation of an FIR filter is:

$$y(n) = \sum_{k=0}^{N-1} h(k)\, x(n-k)$$

where:

$y(n)$ = output signal
$x(n)$ = input signal
$h(k)$ - Filter coefficients
N = No. of filter coefficients (filter order + 1)

FIR LOW PASS FILTER (LPF):

Passband - Frequencies below the cutoff are transmitted with minimal attenuation.
Transition Band - Intermediate region where the response gradually changes.
Stopband - High-frequency components are heavily

attenuated.

FIR HIGH PASS FILTER (HPF):

The FIR HPF consists of three important frequency regions:

Passband for high frequencies.
Transition region
stopband for low frequencies.

FIR BAND PASS FILTER (BPF):

The frequency response contains
- Lower stopband
- Lower transition band
- Passband
- Upper transition band
- Upper stopband.

FIR BAND STOP FILTER (BSF):

The frequency response contains
- Lower passband
- Lower transition band
- stopband
- Upper transition band
- Upper passband

FIR.C:

```
#include <std.h>
#include <c6x.h>
#include "CODEC_Def.h"
#include "LPF2000.cof"
#define GPEN *(volatile unsigned int*)0x01B00000
#define GPDIR *(volatile unsigned int*)0x1B00004
#define GPVAL *(volatile unsigned int*)0x01B00008
#define GP6 0x00000040
#define GP7 0x00000080
#define MUX1 *(volatile unsigned int*)0x19C0000
#define MUX1 *(volatile unsigned int*)0x19C0004
#define INT10_BIT 0x0400
int yn = 0;
short dly[N];
extern float h[N];
void delay(int);
void Configure_AIC23(void);
void main()
{
 short i;
 GPEN = GP6 | GP7;
 GPDIR = GP6;
for(i=0; i<N; i++)
     dly[i] = 0;
Configure_AIC23();
ISTP = 0x00000000;
```

```
ICR = 0X0000FFFD;
MUXH = (MUXH & (~0X1f)) | 0X0f;
IER |= INT10_BIT;
while(1)
{
  GPVAL |= GP6;
  delay (1000);
  GPVAL &= ~GP6;
  delay (1000);
 }
}
void Configure_AIC23 (void)
{
  initialise_McBSP0_SPI();
  initialise_McBSP1_AUDIO();
  Write_SPI_Single (RESET_AIC23, 0);
  Write_SPI_Single (POWER_DOWN, PWR_DOWN_DFLT);
  Write_SPI_Single (VOLUME_L, VOLUME_L_DFLT);
  Write_SPI_Single (VOLUME_R, VOLUME_R_DFLT);
  Write_SPI_Single (VOLUME_HPL, VOLUME_HPL_DFLT);
  Write_SPI_Single (VOLUME_HPR, VOLUME_HPR_DFLT);
  Write_SPI_Single (ANLG_AUDIO_PATH, ANLG_PATH_DFLT);
  Write_SPI_Single (DGTL_AUDIO_PATH, DGTL_PATH_DFLT);
  Write_SPI_Single (DGTL_AUDIO_PATH, DGTL_FMT_DFLT);
  Write_SPI_Single (SAMPLE_RATE, SAMPLE_8KHZ);
  Write_SPI_Single (DGTL_IF_ACTI, DGTL_IF_ACT);
  delay (100);
}
```

```
void delay (period)
{
 int i,j;
 for(i=0; i< period ; i++)
 {
   for(j=0; j<period>>1; j++)
 }
}
interrupt void c_int10(void)
{
 short i;
 int Rdata;
 Rdata = Read_AIC23();
 Rdata = Read_AIC23();
 dly[0] = Rdata;
 yn = 0;
 for(i=0; i<N; i++)
    yn+= (h[i] * dly[i]);
 for(i=N-1; i>0; i--)
    dly[i] = dly[i-1];
 Write_AIC23(yn);
 Write_AIC23(yn);
 ICR = IFR & INT10_BIT;
 return;
}
```

Expt. No.
Date

Page No. 123

PROCEDURE:

1. Connect the TMS320C6713 DSP Starter kit to the computer using a USB cable and switch on the power supply.

2. Launch Code Composer Studio (CCS) and create a new project for the TMS320C6713 DSP board.

3. Open MATLAB and specify the FIR filter design parameters such as sampling frequency, filter order, cutoff frequency, and filter type.

4. Design the required FIR Low Pass, High Pass, Band Pass, or Band Stop filter using the fir1() function or Filter Designer (FDATool).

5. Generate the FIR filter coefficients and verify the magnitude and phase responses in scopefir software.

6. Copy the generated filter coefficients into the coefficient file used by the CCS project.

7. Add the main source file (Fir.c), header

file, coefficient file, linker command file, and required DSP support library files to the project.

8. Write or include the program to initialize the TMS320C6713 DSP board, configure the TLV320AIC23 audio codec, and implement the FIR filtering algorithm using the generated coefficients.

9. Save, build, and compile the project to generate the executable (.out) file.

10. Load the executable (.out) file into the TMS320C6713 DSP board through Code Composer Studio.

11. Connect an audio signal or function generator to the audio input of the DSP board.

12. Run the program so that the DSP processor performs real-time FIR filtering on the incoming signal.

13. Observe the filtered output through the audio output or by displaying the waveform in the CCS Graph window.

14. Verify that the Low Pass Filter passes low-frequency components while attenuating high-frequency components.

15. Verify that the High Pass Filter passes high-frequency components while attenuating low-frequency components.

16. Verify that the Band Pass Filter passes only the specified range of frequencies.

17. Verify that the Band stop Filter attenuates the specified frequency band while allowing all other frequencies to pass.

18. Record the input waveform, output waveform, and frequency response for each filter type.

19. Analyse the filtering performance and verify that the practical output matches the theoretical response.

20. Conclude the experiment by observing the effectiveness of the FIR filter implemented on the TMS320C6713 DSP starter kit.

RESULT:

Thus the FIR filters (low Pass, High Pass, Band Pass, and Band stop) were succesfully designed, implemented, and demonstrated on the TMS320C6713 DSP Starter Kit. The observed outputs matched the expected frequency-selective behavior, confirming the correct operation of each FIR filter type and validating the real-time implementation on the DSP Processor.

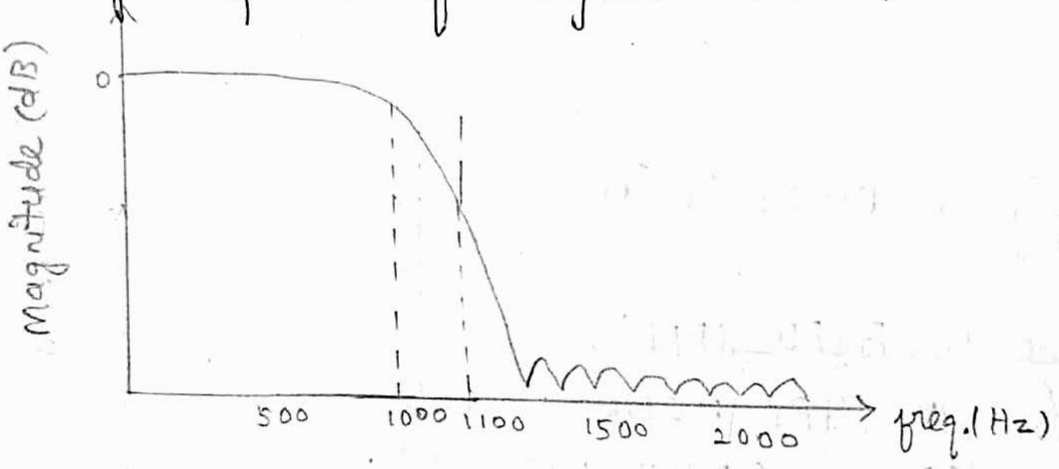
OUTPUT:

FIR Low Pass Filter:

Passband Frequency = 1000 Hz

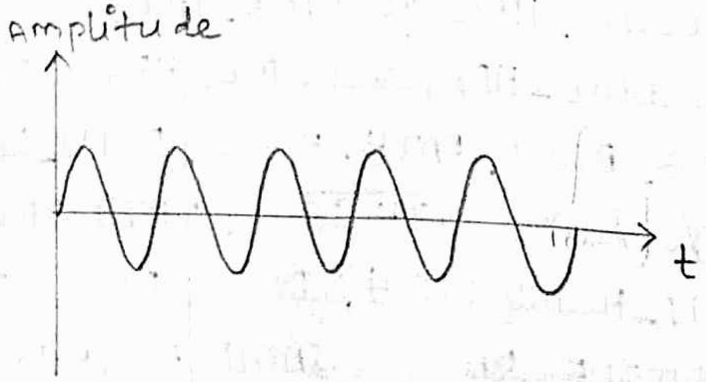
Stopband Frequency = 1100 Hz

Frequency response of designed FIR LPF:

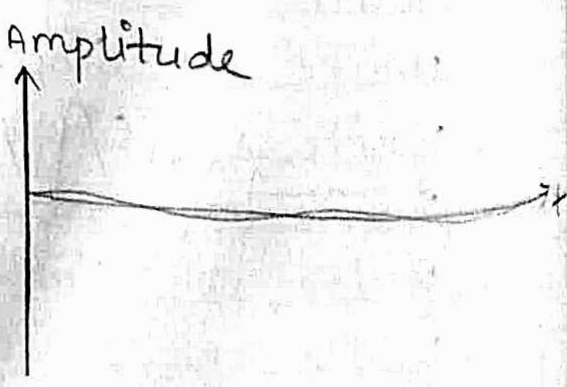


output sine wave for different input frequencies:

a) Input = 500 Hz (Passband)



b) Input = 1500Hz (stopba

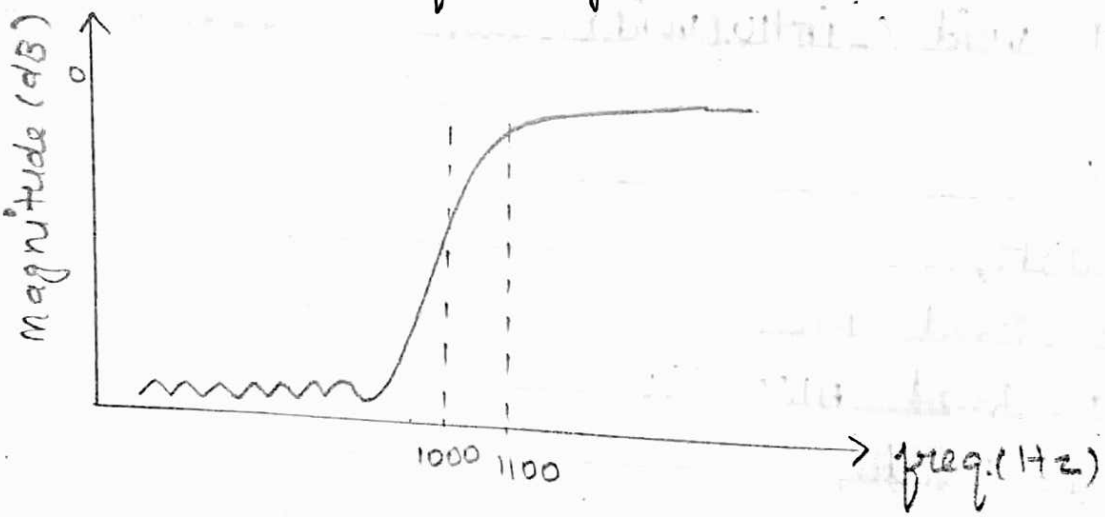


FIR HIGH PASS FILTER:

Passband frequency = 1000 Hz

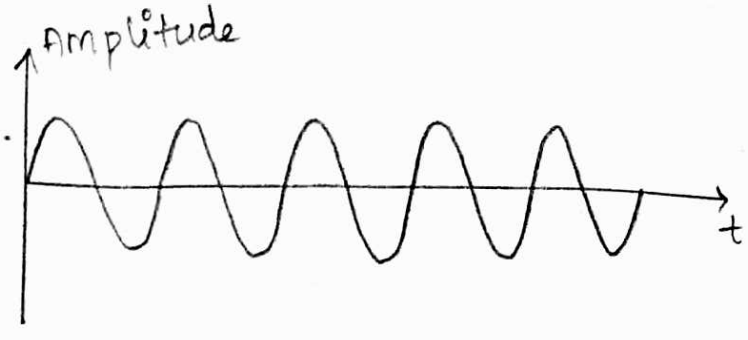
Stopband frequency = 1000 Hz

Frequency response of designed FIR HPF:

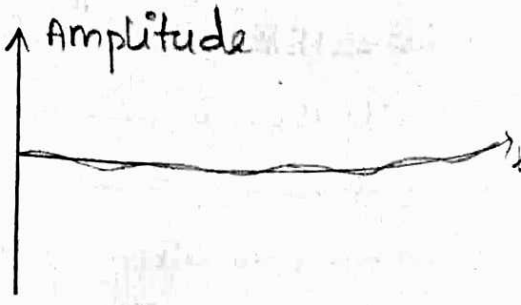


output sine wave for different input frequencies:

a) Input = 1500 Hz
(passband)



b) Input = 800 Hz
(stopband)

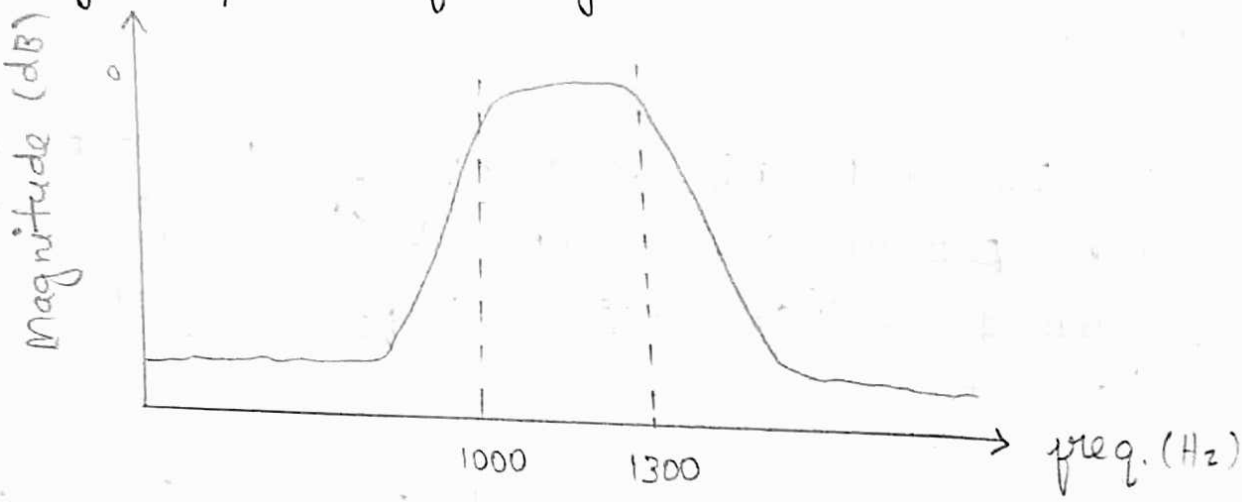


FIR BAND PASS FILTER:

Passband frequency : 1000Hz - 1300Hz

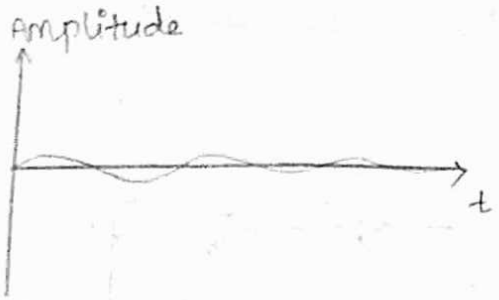
Stopband frequency : >1300Hz & <1000Hz

Frequency response of designed FIR BPF:

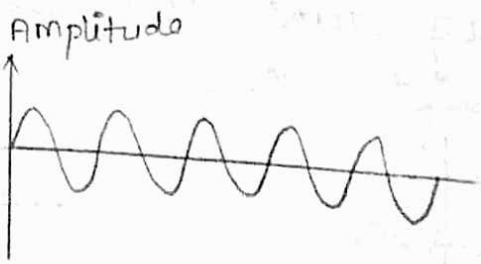


output Sine wave for different input frequencies:

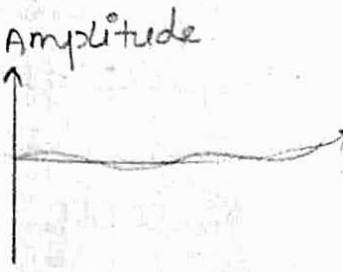
a) Input = 700Hz
(stopband)



b) Input = 1200Hz
(passband)



c) Input = 1500Hz
(Stopband)

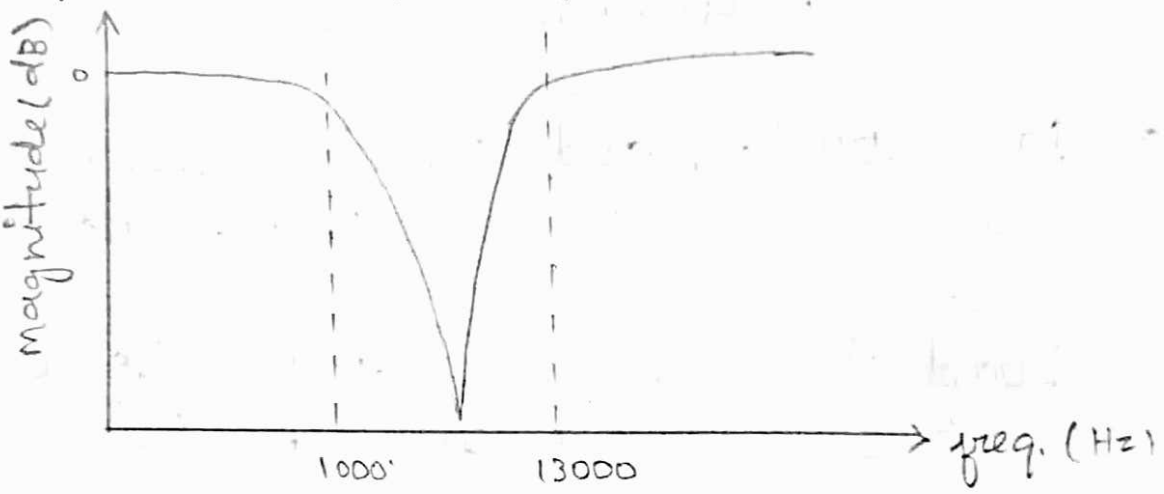


FIR BAND STOP FILTER:

Passband frequency => 13**0**0 Hz and < 1000Hz

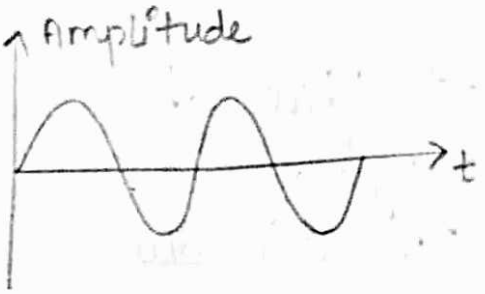
Stopband frequency = 1000 Hz - 1300 Hz

Frequency response of designed FIR BSF:

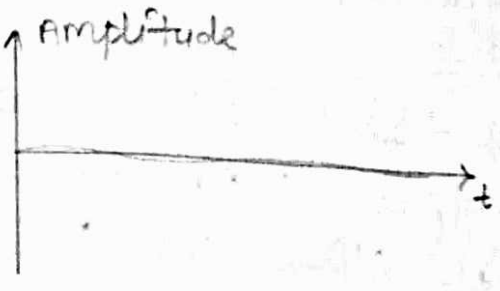


Output sine wave for different input frequencies:

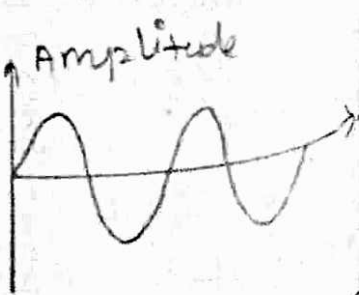
a) Input = 500Hz (passband)



b) Input = 1100Hz (stopband)



c) Input = 1700Hz (passband)



Expt. No. 8
Date 22.9.26

Page No. 131

DEMONSTRATION OF BUTTERWORTH AND CHEBYSHEV IIR FILTER FOR LOW PASS, HIGH PASS, BAND PASS AND BAND STOP FILTERING

AIM:

To design and implement Butterworth and chebyshev Type-1 IIR digital filters such as Low Pass Filter (LPF), High Pass Filter (HPF), Band Pass Filter (BPF), and Band Stop Filter (BSF) using MATLAB, and to analyse and compare their frequency responses, magnitude characteristics, and filtering performance.

HARDWARE REQUIREMENTS:

=> PC / Laptop
=> TMS320C6713 DSP Starter Kit
=> Connecting Cables

SOFTWARE REQUIREMENTS:

=> MATLAB R2020a or later
=> Code Composer Studio (3.1)
=> Scopefir
=> Topview DSP Debugger

THEORY:

1. BUTTERWORTH LOW PASS FILTER (LPF):

A Butterworth Low Pass Filter is an IIR digital filter that allows signals with frequencies below a specified cutoff frequency to pass while attenuating frequencies above the cutoff frequency. It is known as a maximally flat filter because its passband has no ripples, providing a smooth and uniform frequency response.

Characteristics:

1. Maximally flat passband
2. No passband ripple
3. Smooth frequency response
4. Stable recursive filter
5. Moderate transition width

2. BUTTERWORTH HIGH PASS FILTER (HPF):

A Butterworth High Pass Filter (HPF) allows frequencies above the cutoff frequency to pass while attenuating low-frequency components and DC signals.

Characteristics:

=> Smooth passband
=> No ripple
=> Stable operation
=> Gradual attenuation below cutoff frequency
=> Good phase response.

3. BUTTERWORTH BAND PASS FILTER (BPF):

A Butterworth Band Pass Filter allows only a selected range of frequencies to pass while attenuating frequencies below and above that range.

Characteristics:

=> Flat passband
=> No ripple
=> Smooth frequency response
=> Stable operation
=> Moderate transition band

4. BUTTERWORTH BAND STOP FILTER (BSF):

A Butterworth Band Stop Filter rejects frequencies within a specified band while allowing frequencies below and above the stopband to pass.

Characteristics:

=> Smooth passband
=> No ripple
=> Stable operation
=> Moderate stopband attenuation
=> Linear and predictable response.

5. CHEBYSHEV LOW PASS FILTER (LPF):

A Chebyshev Low Pass Filter is an IIR filter that allows low-frequency signals to pass while attenuating high-frequency signals.

Characteristics:

=> Stable operation
=> Equal ripple passband
=> Steeper roll-off
=> Better selectivity

CHEBYSHEV HIGH PASS FILTER (HPF):

A Chebyshev High Pass Filter passes frequencies above the cutoff frequency while attenuating low-frequency components. It provides a faster transition from stopband to passband than a Butterworth HPF due to the presence of passband ripple.

Characteristics:

⇒ Equal ripple passband.
⇒ Sharp cutoff
⇒ Faster roll-off
⇒ Stable operation
⇒ High selectivity

CHEBYSHEV BAND PASS FILTER (BPF):

A Chebyshev Band Pass Filter passes only frequencies within a specified frequency band while attenuating frequencies outside the band.

Characteristics:

=> Equal ripple passband
=> Sharp transition
=> High selectivity
=> Stable recursive filter
=> Narrow transition band

8. CHEBYSHEV BAND STOP FILTER (BSF):

A Chebyshev Band stop Filter attenuates frequencies within a specified stopband while allowing frequencies outside the stopband to pass.

Characteristics:

=> Equal ripple passband.
=> Sharp stopband transition.
=> Better attenuation.
=> Stable operation.
=> High selectivity.

IIR.c :

```
#include<std.h>
#include<clx.h>
#include "CODEC_Def.h"

//#include "LPF_Butterworth.cof"
//#include "LPF_Chebyshev.cof"
//#include "HPF_Butterworth.cof"
#include "HPF_Chebyshev.cof"
//#include "BPF_Butterworth.cof"
//#include "BPF_Chebyshev.cof"
//#include "BSF_Butterworth.cof"
//#include "BSF_chebyshev.cof"
#define GPEN *(volatile unsigned int*)0x01B00000
#define GPDIR *(volatile unsigned int*)0x01B00004
#define GPVAL *(volatile unsigned int*)0x01B00008
#define GP6 0x00000040
#define GP7 0x00000080
#define MUXH *(volatile unsigned int*)0x19C0000
#define MUXL *(volatile unsigned int*)0x19C0004
#define INT10_BIT 0x0400
float x0=0, x1=0, x2=0, x3=0;
float y0=0, y1=0, y2=0, y3=0;
void delay(int);
void Configure_AIC23(void);
void main()
```

```
{
  GPEN = GP6 | GP7;
  GPDIR = GP6;
  Configure_AIC23();
  ISTP = 0x00000000;
  ICR = 0x0000FFF0;
  MUXH = (MUXH & (~0x1f)) | 0x0f;
  IER |= INT10_BIT;
while (1)
{
  GPVAL | = GP6;
  delay (1000);
  GPVAL & = ~GP6;
  delay (1000);
 }
}
void Configure_AIC23 (void)
{
  initialise_McBSP0_SPI();
  initialise_McBSP1_AUDIO();
  write_SPI_single (RESET_AIC23,0);
  Write_SPI_single (POWER_DOWN, PWR_DOWN_DFLT);
  Write_SPI_single (VOLUME_L, VOLUME_L_DFLT);
  Write_SPI_single (VOLUME_R, VOLUME_R_DFLT);
  Write_SPI_single (VOLUME_HPL, VOLUME_HPL_DFLT);
  Write_SPI_single (VOLUME_HPR, VOLUME_HPR_DFLT);
  Write_SPI_single (ANLG_AUDIO_PATH, ANLG_PATH_DFLT);
```

```
Write_SPI_Single (DGTL_AUDIO_PATH, DGTL_PATH_DELT);
Write_SPI_Single (DGTL_AUDIO_FMT, DGTL_FMT_DELT);
Write_SPI_Single (SAMPLE_RATE, SAMPLE_48KHz);
Write_SPI_single ( DGTL_IF_ACTI, DGTL_IF_ACT);
delay(100);
}
void delay(period)
{
int i,j;
for (i=0; i<period; i++)
{
    for(j=0; j<period>>1; j++);
}
}
interrupt void c_int10(void)
{
xo = (short) Read_AIC23();
xo = (short)Read_AIC23();
yo = xo*no + x1*n1 + x2*n2 + x3*n3 - y1*d1 - y2*d2
                                   - y3*d3;
x3 = x2;
x2 = x1;
x1 = xo;
y3 = y2;
y2 = y1;
y1 = yo;
Write_AIC23((short)(yo));
```

```
Write_AIC23((short)(yo));
ICR = IFR & INTIO_BIT;
return;
}
```

PROCEDURE:

1. Connect the TMS320C6713 DSP Starter Kit to the computer using a USB cable and switch on the power supply.

2. Launch Code composer Studio and create a new project for the TMS320C6713 DSP board.

3. Open MATLAB and specify the filter design parameters, including sampling frequency, filter order, cutoff frequency, passband ripple and stopband attenuation.

4. Design the required Butterworth or Chebyshev IIR filter (Low Pass, High Pass, Band Pass, or Band Stop) using MATLAB.

5. Generate the numerator and denominator filter coefficients and verify the filter response using the freqz() function or filter designer.

Expt. No. .....
Date .....

Page No. .....

6. Copy the generated filter coefficients into the coefficient file used in the CCS project.

7. Add the main source file, header files, coefficient files, linker command file, and required support library files to the project.

8. Write or include the program to initialize the DSP board, configure the TLV320AIC23 audio codec, and implement IIR filter.

9. Save, build, and compile the project successfully.

10. Load the generated executable (.out) file into the TMS320C6713 DSP board through Code Composer Studio.

11. Apply an audio signal or function generator signal to the audio input of the DSP board.

12. Run the program so that the DSP processor performs real-time filtering of the input signal.

13. Observe the filtered output through the audio output or display the waveform using the Graph Window in Code Composer Studio.

14. Verify that the Low Pass Filter passes low-frequency components while attenuating high-frequency components.

15. Verify that the High Pass Filter passes high-frequency components while attenuating low-frequency components.

16. Verify that the Band Pass Filter passes only the specified frequency band.

17. Verify that the Band Stop Filter passes frequencies other than the specified frequency band.

18. Repeat the procedure for both Butterworth and Chebyshev filters and compare their responses.

19. Record the input waveform, output waveform, magnitude response, phase response and observations.

20. Analyze the results, compare the characteristics of Butterworth and Chebyshev filters and conclude the experiment.

RESULT:

Thus the Butterworth and Chebyshev Type-I IIR filters for Low Pass, High Pass, Band Pass and Band Stop were successfully designed and analysed using MATLAB. The frequency responses and filtering characteristics of both filter families were compared, verifying their suitability for different digital signal processing applications.

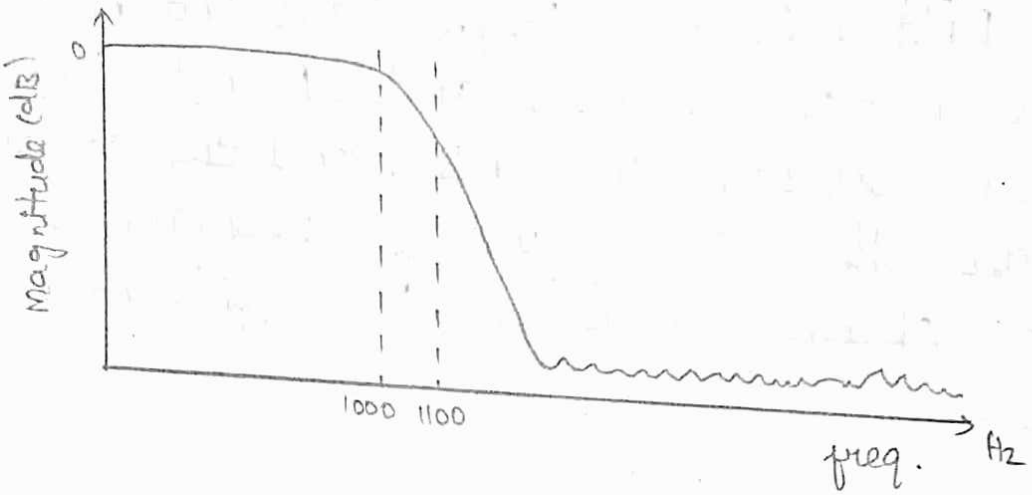
OUTPUT:

BUTTERWORTH IIR LOW PASS FILTER:

Passband frequency = 1000Hz

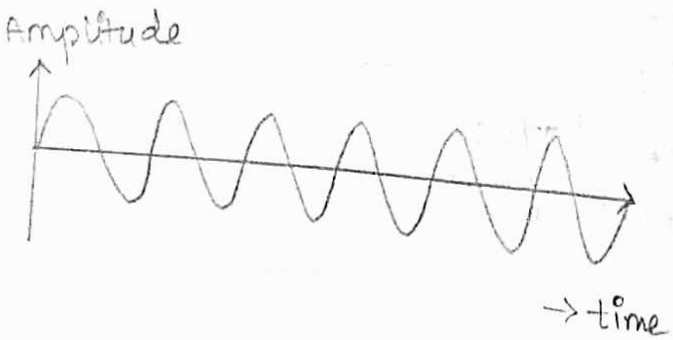
Stopband frequency = 1100Hz

Frequency Response:

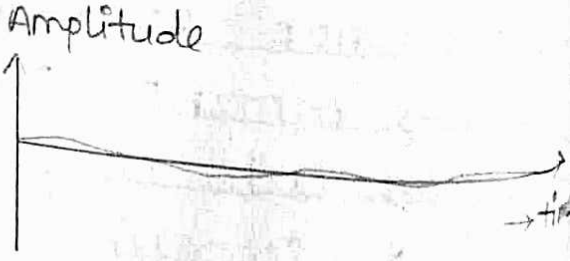


Output Sine wave for different input frequencies:

a) Input = 500Hz (Passed)



b) Input = 1500Hz (attenuate

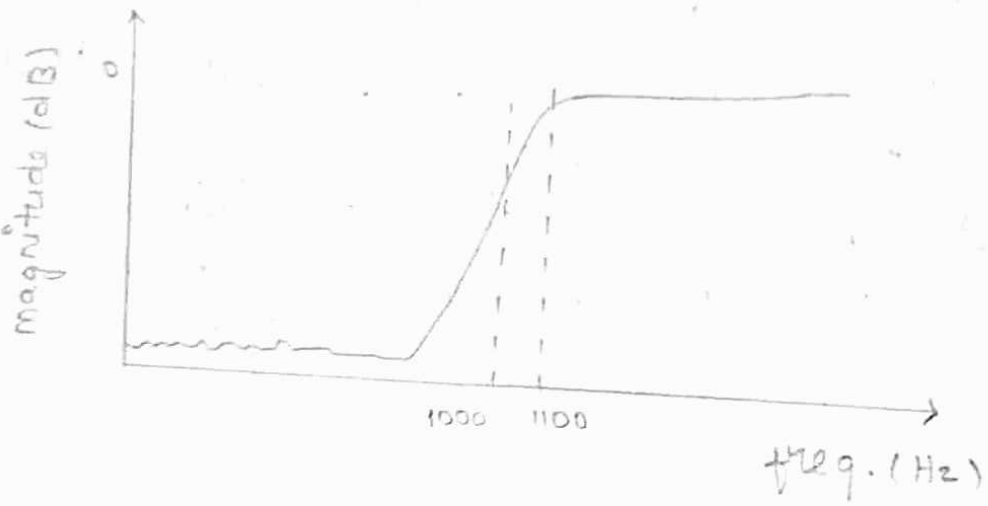


BUTTERWORTH IIR HIGH PASS FILTER:

Passband frequency = 1100Hz

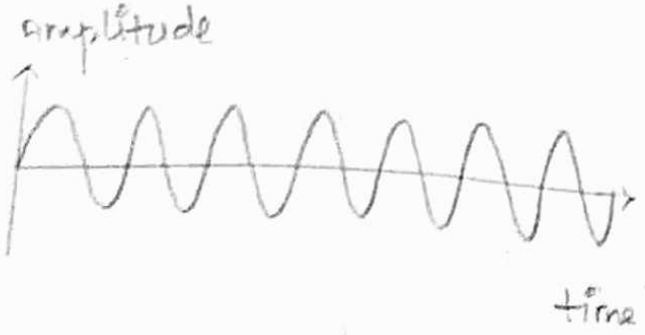
Stopband frequency = 1000 Hz

Frequency Response.

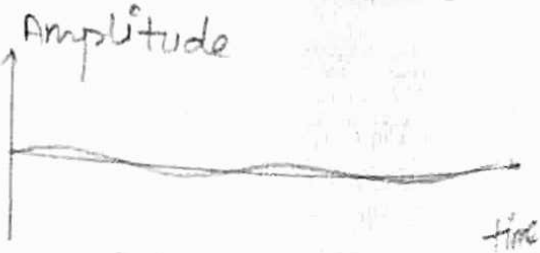


output sine wave for different input frequencies.

a) Input = 1500Hz (passed)



b) Input = 800Hz (attenuated)

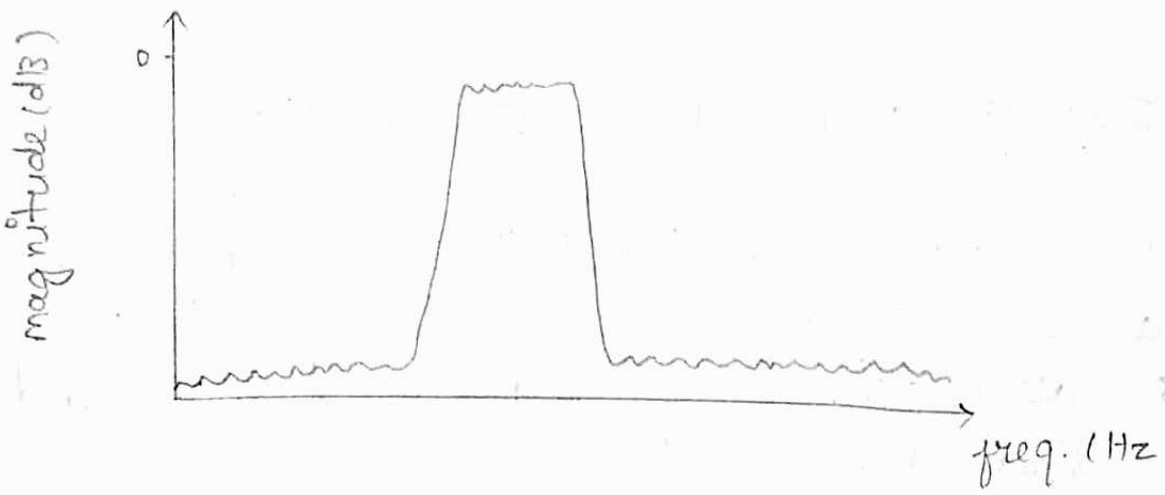


BUTTERWORTH IIR BAND PASS FILTER:

Passband = (1000 - 1300) Hz

Stopband = <1000 Hz & >1300 Hz

Frequency Response:

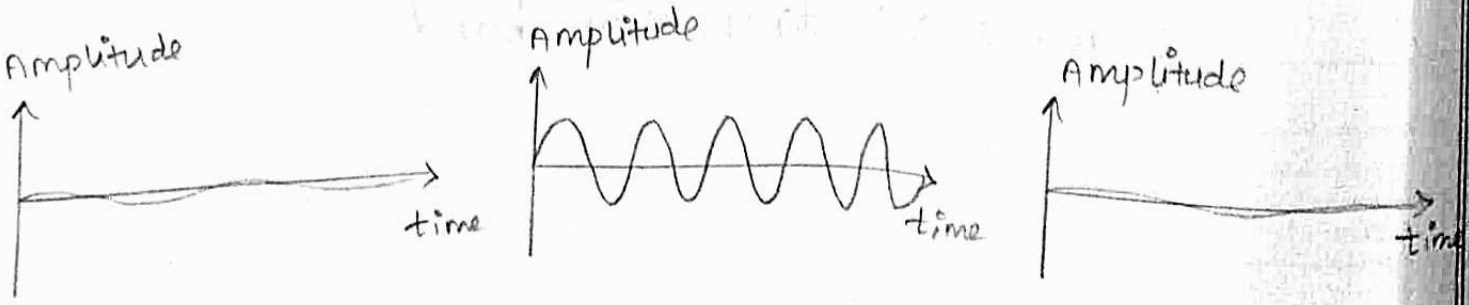


output sine wave for different input frequencies:

a) Input = 800Hz (attenuated)

b) Input = 1200 Hz (passed)

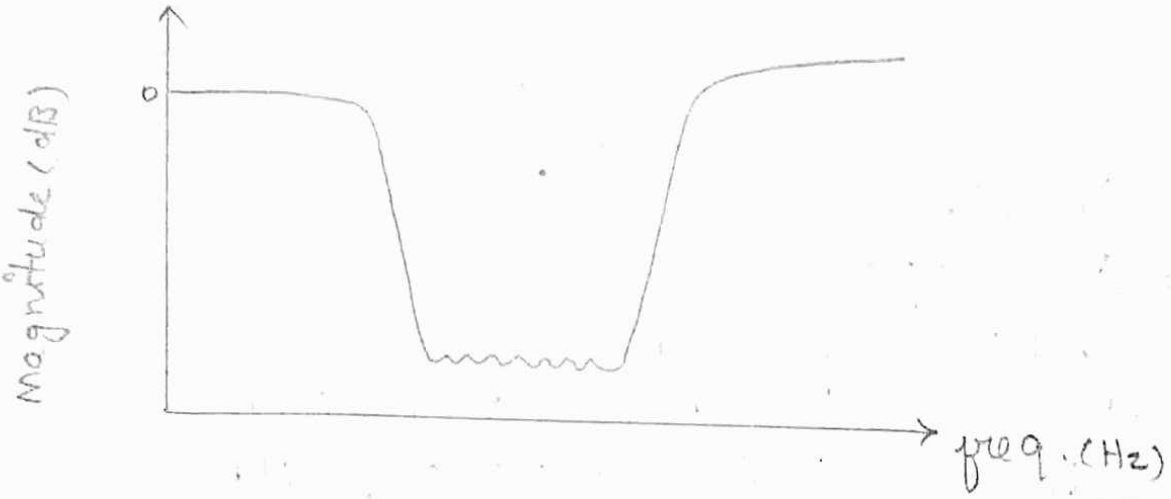
c) Input = 1700Hz (attenuated)



BUTTERWORTH IIR BAND STOP FILTER:

Passband = < 1000 & > 1300Hz

stopband = (1000 - 1300)Hz

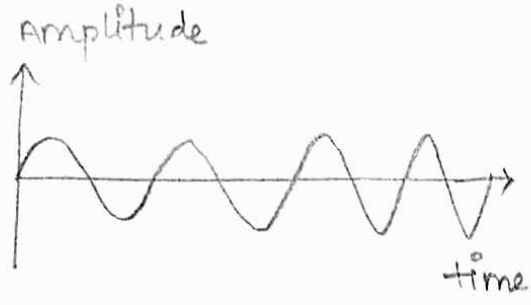


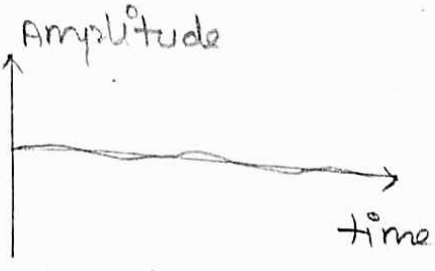
output sine wave for different input frequencies:

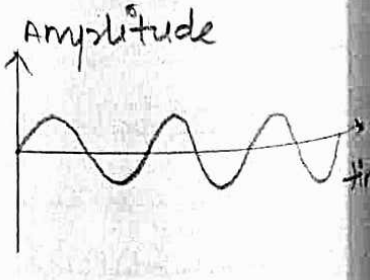
a) Input = 500Hz (passed)

b) Input = 1100Hz (attenuated)

c) Input = 2500Hz (passed)





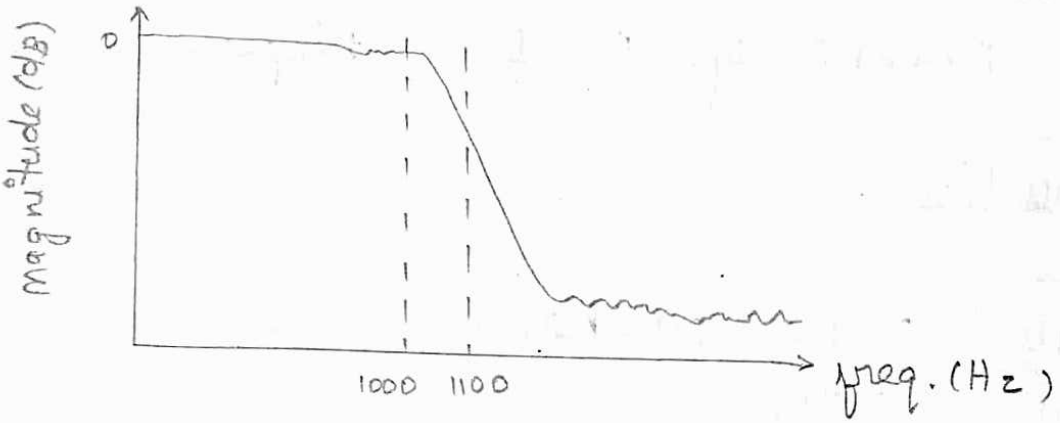


CHEBYSHEV TYPE-1 IIR LOW PASS FILTER:

Passband edge frequency = 1000Hz

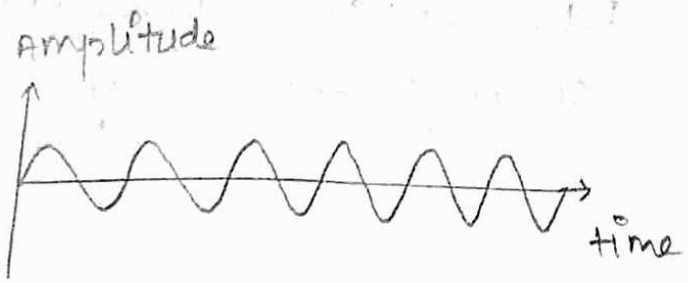
Stopband edge frequency = 1100Hz

Frequency Response:

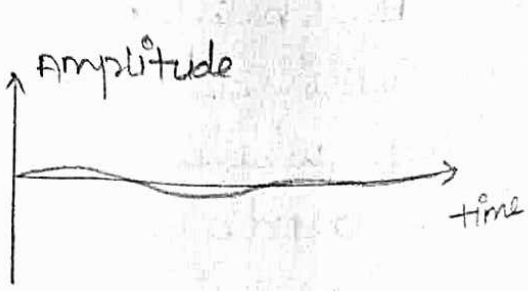


output sine wave for different input frequencies:

a) Input = 900Hz
(passed)



b) Input = 1500 Hz
(attenuated)

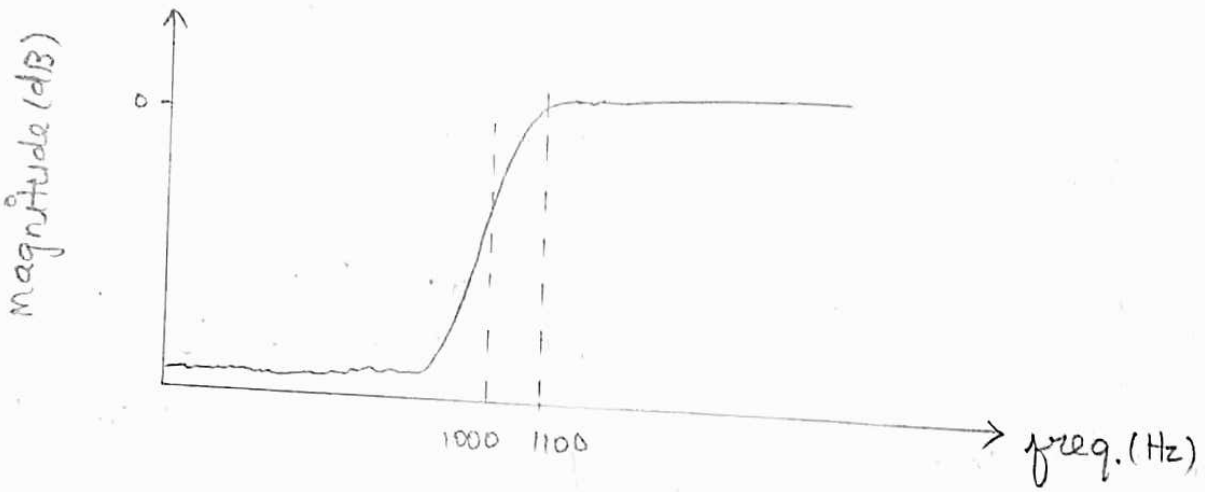


CHEBYSHEV TYPE-I IIR HIGH PASS FILTER:

Passband edge frequency = 1100 Hz

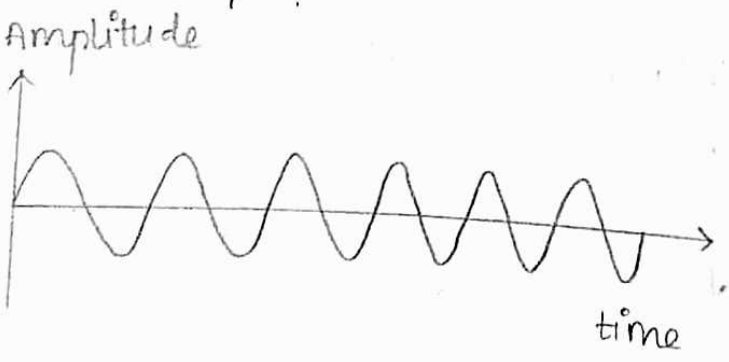
Stopband edge frequency = 1000 Hz

Frequency Response:

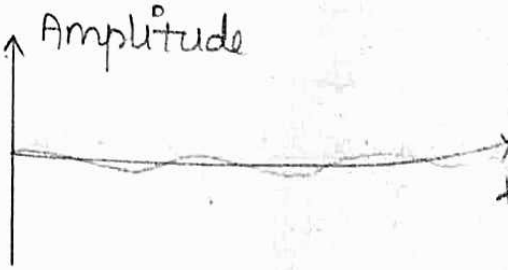


output sine wave for different input frequencies:

a) Input = 2000 Hz
(passed)



b) Input = 800 Hz
(attenuated)

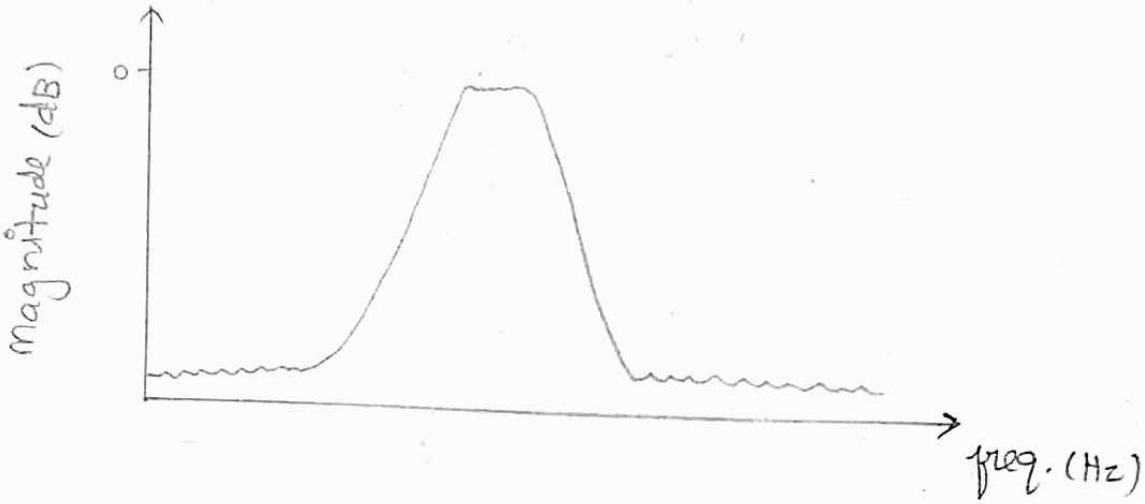


CHEBYSHEV TYPE-1 IIR BAND PASS FILTER:

Passband = (1000 - 1300) Hz

Stopband = < 1000 Hz & > 1300 Hz

Frequency Response:

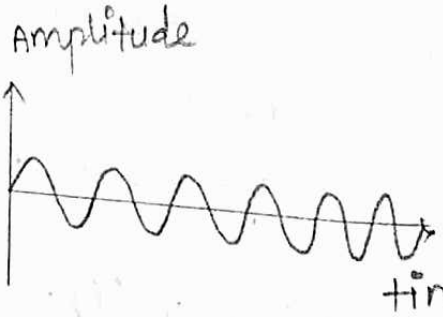


Output sine wave for different input frequencies:

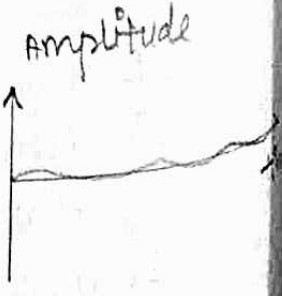
a) Input = 500 Hz (attenuated)



b) Input = 1200 Hz (passed)



c) Input = 1500 Hz (attenuated)

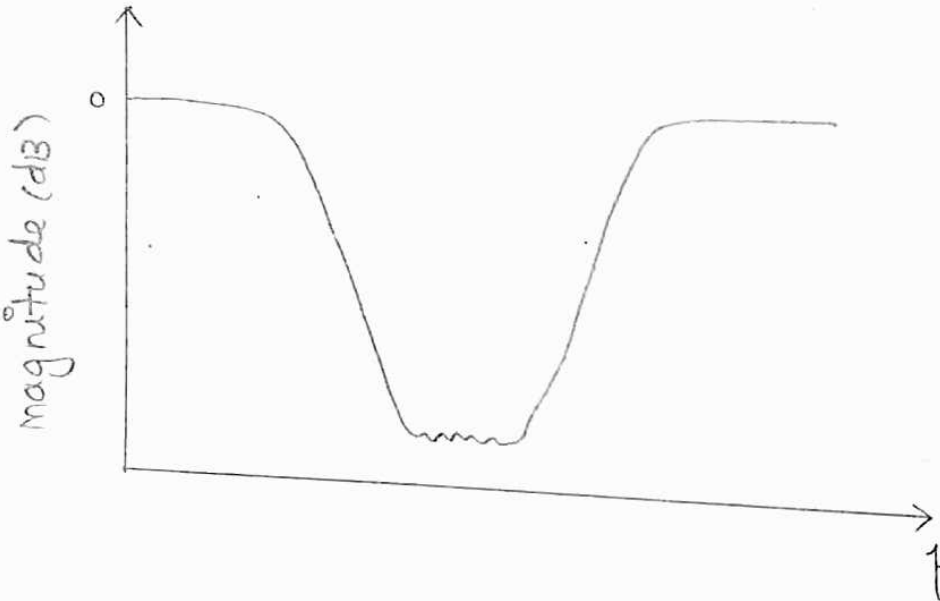


CHEBYSHEV TYPE-1 IIR BAND STOP FILTER:

Passband - > 1300Hz and < 1000 Hz

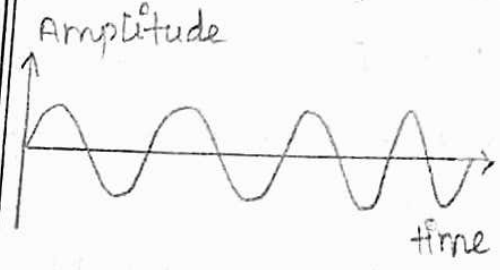
Stopband - (1000 - 1300) Hz

Frequency Response:

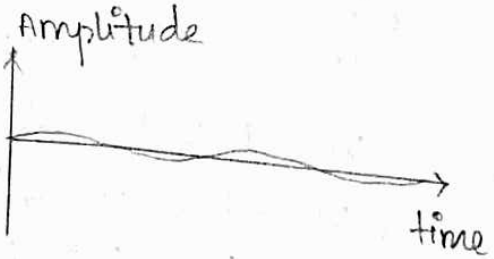


output sine wave for different input frequencies:

a) Input = 500Hz (passed)



b) Input = 1100Hz (attenuated)



c) Input = 2000 (passed)

